

**AUTISM PREDICTION USING MACHINE LEARNING:
A COMPARATIVE ANALYSIS OF CLASSIFICATION
ALGORITHMS**

A dissertation submitted in partial fulfillment of the requirements for

the award of the degree of

MASTER OF SCIENCE IN COMPUTER SCIENCE

By

S. MOHAMED ISMAIL

24SMS17

Under the guidance of

Mr. K. GANESH KUMAR, M.C.A., M. Phil., NET.,

Assistant Professor, PostGraduate

Department of Computer Science,

Sadakathullah Appa College (Autonomous) Tirunelveli – 627 011



POSTGRADUATE DEPARTMENT OF COMPUTER SCIENCESADAKATHULLAH

APPA COLLEGE (AUTONOMOUS)

TIRUNELVELI – 627 011

APRIL – 2026

CERTIFICATE

Certified that the dissertation work with the title, “**AUTISM PREDICTION USING MACHINE LEARNING: A COMPARATIVE ANALYSIS OF CLASSIFICATION ALGORITHMS**” submitted by **S. MOHAMED ISMAIL** with the register number 24SMS07 in partial fulfillment of the requirements for the award of the degree of **Master of Science in Computer Science at the PostGraduate Department of Computer Science, Sadakathullah Appa College (Autonomous)**, is a work done by the candidate during the period of 2025-2026, under my guidance and supervision and this dissertation or any part thereof has not been submitted elsewhere for any other Degree or Diploma.

Tirunelveli – 627 011

DATE:

INTERNAL GUIDE

Mr. K. GANESH KUMAR, M.C.A., M. Phil., NET.,
Assistant Professor,
PostGraduate Department of Computer Science,
Sadakathullah Appa College, (Autonomous),
Rahmath Nagar,
Tirunelveli – 627 011.

HEAD OF THE DEPARTMENT

Dr. V. ROSELINE, M.Sc., M.Phil., B.Ed, SET., NET., P.hD.,
Head & Assistant Professor,
PostGraduate Department of Computer Science,
Sadakathullah Appa College (Autonomous),
Rahmath Nagar,
Tirunelveli – 627 011

S. MOHAMED ISMAIL,
Reg. No. 24SMS17,
M.Sc. Computer Science,
PostGraduate Department of Computer Science,
Sadakathullah Appa College (Autonomous),
Rahmath Nagar, Tirunelveli – 627 011.

DECLARATION BY THE CANDIDATE

I hereby declare that, the dissertation with the title “**AUTISM PREDICTION USING MACHINE LEARNING: A COMPARATIVE ANALYSIS OF CLASSIFICATION ALGORITHMS**” submitted in partial fulfillment of the requirements for the award of the degree of **Master of Science in Computer Science at the PostGraduate Department of Computer Science, Sadakathullah Appa College (Autonomous)**, is my original work done under the guidance of **Mr. K. GANESH KUMAR, M.C.A., M. Phil., NET., Assistant Professor, PostGraduate Department of Computer Science, Sadakathullah Appa College (Autonomous), Tirunelveli-11** and this work has not been submitted elsewhere for any other Degree or Diploma.

Tirunelveli – 627 011

(Signature of the Candidate)

(Signature of the Guide)

Examiner 1:

Examiner 2:

ACKNOWLEDGEMENT

“It is not possible to prepare a project report without the assistance and encouragement of other people. This one is certainly no exception.”

On the very outset of this report, I would like to extend my sincere and heartfelt obligation towards all the personages who have helped me in this endeavor. Without their active guidance, help, cooperation & encouragement, I would not have made headway in the project.

I thank my parents, for their unflagging belief that despite their incomprehension about what I do, I am indebted to my parents for inculcating in me the dedication and discipline to do whatever I undertake well.

I feel immense pleasure to express my heart-felt gratitude to our Principal **Dr. S. M. ABDUL KADER, M.Sc., M. Phil., PGDCA., SLET., Ph. D.**, for his support in carrying this project in all aspects. I also thank to our Vice Principal **Dr. R. JANET RANI, MSc (MB)., MSc (Bot)., MPhil., PhD.**, for her constant support towards this project.

I would like to express my heartfelt gratitude to our respected Head of the Department **Dr. V. ROSELINE, M.Sc., M.Phil., B.Ed, SET., NET., P.hD.**, for her constant support, valuable guidance and encouragement throughout the successful completion of this project.

I would like to express my sincere and heartfelt gratitude to, **Mr. K. GANESH KUMAR, M.C.A., M. Phil., NET.**, for his constant support, valuable expert guidance, and encouragement throughout the course of this project. His insightful suggestions and motivation played a crucial role in the successful completion of my project titled “**AUTISM PREDICTION USING MACHINE LEARNING: A COMPARATIVE ANALYSIS OF CLASSIFICATION ALGORITHMS**”.

My sincere thanks to all the staff members of the department for their constant support and concerning wishes that motivated and made to grow in bigger dimension. At last but not scomeplete this project.

ABSTRACT

Autism Spectrum Disorder (ASD) is a complex neurodevelopmental condition that affects communication, behaviour, and social interaction. Early and accurate detection of ASD is critical for timely intervention and improved long-term outcomes. This project presents a comprehensive comparative study of eight machine learning classification algorithms — Logistic Regression, Support Vector Machine (SVM), Decision Tree, Random Forest, AdaBoost, XGBoost, LightGBM, and CatBoost — applied to the Autism Screening Dataset sourced from Kaggle.

The dataset comprises 800 records with 22 attributes including AQ-10 behavioural questionnaire responses, demographic features, and medical history. A structured machine learning pipeline is implemented involving data preprocessing, feature engineering, feature scaling using MinMaxScaler, and feature selection using SelectKBest with Chi-Square scoring. Novel engineered features — `sum_score`, `ageGroup`, `interaction_score`, and `ind` — are introduced to enhance predictive capability. Class imbalance is addressed using RandomOverSampler to improve minority class detection.

All eight models are evaluated on a held-out test set of 160 samples using Accuracy, Precision, Recall, F1-Score, and AUC-ROC metrics. Experimental results demonstrate that ensemble learning methods consistently outperform traditional classifiers. LightGBM achieves the highest accuracy of 94.38% with an AUC of 0.971, followed by XGBoost (93.13%, AUC 0.962) and Random Forest (92.50%, AUC 0.953). The study confirms that feature engineering and proper handling of class imbalance significantly improve model reliability. The proposed system has strong potential as a decision support tool for early ASD screening in healthcare applications.

Keywords: Autism Spectrum Disorder, Machine Learning, Classification, LightGBM, XGBoost, Feature Engineering, AQ-10, Early Detection, Ensemble Learning.

LIST OF FIGURES

FIGURE NO	FIGURE TITLE	PAGE NO
1.1	Autism Spectrum Disorder Prevalence	3
3.1	Logistic Regression	22
3.2	Support Vector Machine (SVM)	23
3.3	Random Forest	24
3.4	Decision Tree	25
3.5	AdaBoost	26
4.1	System architecture	28
4.2	ML Preprocessing Pipeline Flowcart	29
4.3.1	Input Dataset	32
4.3.2	Handling Missing Values	33
4.3.3	Training and Testing Split	34
4.4.1	Logistic Regression Sigmoid Function	35
4.4.2	Support Vector Machine Algorithm	36
4.4.3	Decision Tree Algorithm	37
4.4.4	Random Forest Algorithm	38
4.4.5	Adaboost Algorithm	39
4.4.6	Xgboost Algorithm	40
4.4.7	LightGBM Leaf-wise Growth	41
4.4.8	CatBoost Algorithms	42
4.5.1	Google Colab Environment	44
4.5.2	Input Dataset	44
4.5.3	Handling Missing Values	45
4.5.4	Target Variable , Analysing Target Variable , Target Output	46
4.5.5	Training and Testing Split	46
4.5.6.1	Logistic Regression Model	47

4.5.6.2	Support Vector Machine Model	48
4.5.6.3	Decision Tree Model	49
4.5.6.4	Random Forest Model	50
4.5.6.5	Adaboost Model	51
4.5.6.6	XGBoost Model	52
4.5.6.7	LightGBM Model	52
4.5.6.8	CatBoost Model	53
5.1	The ROC curves for all eight classifiers	56
5.2	The Model Accuracy Comparison in Bar Chart	58
6.2.1	The Model Accuracy Comparison in Bar Chart	64
6.2.2	The ROC Curves- All 8 Classifiers	64

LIST OF TABLES

TABLE NO	TABLE TITLE	PAGE NO
4.1	Dataset Feature Description	30
4.2	AQ-10 Questionnaire Item	31
5.1	Comparative Performance of All Eight ML Classifiers	57

INDEX

TABLE OF CONTENTS	PAGE NO
ACKNOWLEDGEMENT	iii
ABSTRACT	iv
LIST OF FIGURES	v
LIST OF TABLES	vii
CHAPTER 1 1.1 INTRODUCTION OF THE PROJECT	1
1.2 MOTIVATION	3
1.3 PROBLEM STATEMENT	4
1.4 BACKGROUND	4
1.5 SCOPE AND OBJECTIVES	6
1.6 DOCUMENT OVERVIEW	6
CHAPTER 2 LITERATURE REVIEW	7
2.1 LITERATURE REVIEW	7
CHAPTER 3 SYSTEM SPECIFICATION	12
3.1 HARDWARE REQUIREMENTS	12
3.2 SOFTWARE REQUIREMENTS	12
3.3 SOFTWARE DESCRIPTION	13
CHAPTER 4 METHODOLOGY AND IMPLEMENTATION	28
4.1 SYSTEM ARCHITECTURE	28
4.2 DATASET	28
4.3 METHODOLOGY	30
4.4 EVALUATION OF MACHINE LEARNING MODEL	35
4.5 IMPLEMENTATION	43
CHAPTER 5 EXPERIMENTAL RESUL	54
5.1 PERFORMANCE METRICS	54
5.2 PERFORMANCE EVALUATION	57
CHAPTER 6 SOURCE CODE AND SCREENSHOT	60
6.1 SOURCE CODE	60
6.2 SCREENSHOT	64
CHAPTER 7 CONCLUSION AND FUTURE ENHANCEMENT	65
7.1 CONCLUSION	65
7.2 FUTURE WORK	66
REFERENCES	67

CHAPTER 1

CHAPTER 1

INTRODUCTION

1.1 INTRODUCTION

Autism Spectrum Disorder (ASD) has become an important topic in today's world, especially in the field of healthcare and child development. Over the past few years, the number of autism cases has increased, and many families are affected by it. Autism is not a single condition but a spectrum, meaning it can appear in different forms and levels of severity. Some individuals may have mild symptoms, while others may require more support in their daily lives. One of the main challenges with autism is that it is not always easy to identify in the early stages. In many cases, children may appear normal at first, and the signs become noticeable only as they grow older. Because of this, early detection becomes very important. If autism is identified early, it becomes easier to provide proper guidance, therapy, and support, which can greatly improve the individual's development and quality of life.

The human brain controls communication, behavior, and social interaction. In individuals with autism, the development of these functions may be different compared to others. This difference can lead to challenges in speaking, understanding others, and interacting socially. Some common signs include difficulty in making eye contact, delayed speech, lack of interest in social activities, and repetitive behavior. However, these signs are not always clearly visible in the beginning. In many situations, parents or caregivers may notice small differences but may not recognize them as symptoms of autism. This delay in identifying the condition can affect early intervention, which is very important for better outcomes.

Traditionally, autism is diagnosed by doctors and specialists through observation and structured assessments. These include behavioral analysis, interviews with parents, and various screening tools. While these methods are reliable, they require trained professionals and can take a considerable amount of time. In some areas, access to such specialists may also be limited. As a result, there is a growing need for alternative methods that can assist in early detection and make the process faster and more accessible.

With the advancement of technology, especially in the field of data science, large amounts of healthcare-related data are now available. This includes screening results, patient records, and behavioral observations. Such data can be very useful if analyzed properly. Machine learning, which is a part of artificial intelligence, provides a way to analyze this data and find meaningful patterns. It allows systems to learn from data and make predictions without being explicitly programmed for every situation. Because of this, machine learning is now widely used in healthcare for tasks such as disease prediction and decision support.

Most of the existing machine learning models used for autism prediction are based on supervised learning. In this approach, the model is trained using labeled data, where each record is already classified as autistic or non-autistic. The model learns from this data and then predicts the outcome for new cases. While this method can produce good results, it depends heavily on the availability of labeled data. In real-world situations, collecting and labeling medical data is not always easy. It requires expert knowledge, time, and proper validation. There are also concerns related to privacy and data security. Because of these reasons, the availability of high-quality labeled datasets is often limited.

To improve the performance of machine learning models, it is important to prepare the data properly. This includes steps such as cleaning the data, handling missing values, and converting categorical values into numerical form. Feature engineering is another important step, where new features are created from existing data to provide better insights. For example, combining multiple behavioral scores into a single value can help the model understand overall patterns more clearly. Feature selection techniques are also used to identify the most important features, which helps in reducing complexity and improving accuracy.

In this project, machine learning techniques are used to predict autism using a screening dataset. The dataset contains information such as behavioral responses, personal details, and medical history. Different models are applied to this dataset to study their performance. These models include Logistic Regression, Support Vector Machine, Decision Tree, Random Forest, AdaBoost, XGBoost, LightGBM, and CatBoost. Each of these models works in a different way and has its own advantages.

Before applying these models, the dataset is processed carefully. This includes removing inconsistencies, converting data into the required format, and scaling the features.

Additional features such as the total behavioral score are also created to improve prediction. The dataset is then divided into training and testing sets so that the models can be evaluated properly. Various performance metrics such as accuracy, precision, recall, F1 score, and ROCAUC are used to measure how well the models perform. Confusion matrices are also used to understand how accurately the models classify the data.

By comparing the results of different models, it becomes possible to identify which approach works best for autism prediction. Some models may perform better depending on the type of data and features used. The main aim of this project is to develop a reliable system that can assist in early detection of autism. Such a system can be useful for healthcare professionals, parents, and educators in identifying potential cases at an early stage and taking appropriate actions.

1.2 MOTIVATION

The number of autism cases is increasing, and in many situations, the condition is identified only after noticeable symptoms appear. This delay can affect the development of the individual and make it harder to provide effective support. Early detection plays a key role in improving outcomes, but traditional diagnostic methods are not always easily accessible.

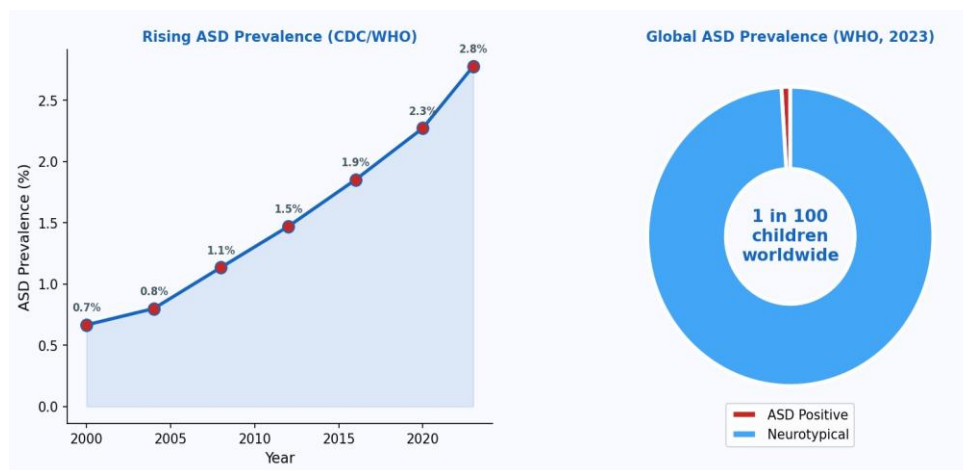


Figure 1.1 Autism Spectrum Disorder Prevalence

Most current prediction methods rely on labeled data, which is not always available. Collecting and preparing such data requires time, effort, and expert involvement. There are also concerns related to privacy when dealing with medical data. Because of these challenges, there is a need for simple and effective approaches that can work with available data.

The motivation behind this project is to explore how machine learning can be used to analyze autism-related data and provide early predictions. By comparing different models, this study aims to find a method that is both accurate and practical. The goal is to support early screening and improve decision-making in healthcare.

1.3 PROBLEM STATEMENT

Autism is a complex condition that is not always easy to detect in its early stages. Traditional diagnostic methods require time and expertise, which may not always be available. Many machine learning approaches depend on labeled datasets, which are difficult to obtain in real-world scenarios.

Another challenge is that autism-related data can be complex and may contain hidden patterns that are not easy to identify. Because of these difficulties, there is a need for an efficient system that can analyze available data and provide reliable predictions.

This project focuses on developing a machine learning-based solution for autism prediction. The aim is to study different models and identify a suitable approach for early detection using screening data.

1.4 BACKGROUND

Autism has been studied for many years, and different methods have been used to understand and diagnose it. Earlier, diagnosis was mainly based on clinical observation and expert judgment. While these methods are still important, they may not always help in identifying autism at an early stage.

With the growth of digital technology, a large amount of healthcare data is now available. This includes various types of patient information that can be used for analysis.

Machine learning has made it possible to use this data effectively. It helps in finding patterns and making predictions based on past information.

Most existing studies use supervised learning methods, which require labeled data. However, due to the limited availability of such data, there is a need to explore better and more practical approaches for autism prediction.

1.4.1 OVERVIEW OF AUTISM

Autism Spectrum Disorder includes a range of conditions that affect communication, behavior, and social interaction. The symptoms can vary widely from one person to another. Some individuals may have mild difficulties, while others may require continuous support.

The exact causes of autism are still being studied, but factors such as genetics and environment may play a role. Early identification is important because it allows for timely intervention. With proper support, individuals with autism can improve their skills and lead better lives.

1.4.2 TRADITIONAL AND MACHINE LEARNING METHODS

Traditional methods rely on clinical assessments and observation. While they are effective, they may not always provide early results. Machine learning offers an alternative by analyzing data and identifying patterns that may not be obvious.

These models can handle multiple features at once and improve prediction accuracy. However, many of them depend on labeled data, which is a limitation in practical situations.

1.4.3 LIMITATIONS OF EXISTING APPROACHES

Many existing methods face challenges such as limited data, missing values, and overfitting. Supervised models require labeled data, which is not always available. These issues can affect the reliability of the predictions.

Because of these limitations, there is a need for improved techniques that can handle real-world data more effectively. To overcome these challenges, robust models with better generalization and efficient data handling techniques must be developed to ensure more accurate and reliable predictions.

1.4.4 MACHINE LEARNING APPROACH

Machine learning provides a flexible way to analyze autism data. Different models use different techniques to learn from the data. Some focus on simple relationships, while others use more complex methods.

By testing multiple models and comparing their results, it is possible to find the most suitable approach for prediction.

1.5 SCOPE AND OBJECTIVES

Scope

This study focuses on predicting autism using machine learning based on available screening data. It includes data preprocessing, model building, and evaluation. The work is limited to analysis using the given dataset and does not involve real-time medical use.

Objectives

- To study autism data using machine learning
- To apply different models for prediction
- To compare model performance
- To identify the best model
- To support early detection

1.6 DOCUMENT OVERVIEW

This report explains the entire project in a clear and organized manner. The introduction provides an overview of autism and the need for early detection. The literature review discusses previous work in this area. The system specification explains the tools used. The methodology describes the steps followed in the project. The results section shows the performance of the models. Finally, the conclusion summarizes the findings and suggests future improvements.

CHAPTER 2

CHAPTER 2

LITERATURE REVIEW

2.1. LITERATURE REVIEW

Autism Spectrum Disorder (ASD) has gained significant attention in recent years due to its increasing prevalence and impact on individuals and society. Researchers across various disciplines have explored different approaches for early diagnosis and prediction of autism. Traditional diagnostic methods rely heavily on behavioral assessments and clinical expertise, which are often time-consuming and require trained professionals. With the rapid advancement of Machine Learning (ML), automated systems have been developed to assist in early autism detection, providing faster and more accessible solutions.

Catherine Lord et al [1]. (2000) introduced the Autism Diagnostic Observation Schedule (ADOS), which is considered one of the most reliable and standardized tools for diagnosing autism. ADOS provides a structured framework for assessing communication skills, social interaction, and behavioral characteristics through direct observation. This method has been widely adopted in clinical settings due to its accuracy and consistency. Similarly, Michael Rutter et al. [1] (2003) developed the Autism Diagnostic Interview-Revised (ADI-R), which involves a comprehensive and structured interview conducted with caregivers to evaluate the developmental history and behavioral traits of individuals. Although ADOS and ADI-R are regarded as gold standard diagnostic tools, they require significant time, trained professionals, and clinical expertise, which limits their accessibility, especially in rural and underdeveloped regions.

The rapid advancement of Artificial Intelligence has transformed the field of healthcare by enabling the development of intelligent diagnostic systems. Machine learning, a subset of artificial intelligence, has shown great potential in identifying hidden patterns in complex datasets and making accurate predictions. Richard O. Duda et al. [2] (2012) emphasized that machine learning algorithms can analyze large volumes of data and detect intricate relationships that are often beyond human capability. This ability is particularly useful in autism prediction, where subtle behavioral and developmental patterns need to be identified at an early

stage. The integration of machine learning in healthcare has led to the development of predictive models that can assist clinicians in making faster and more accurate decisions.

Among the various machine learning algorithms, Logistic Regression has been widely used for medical diagnosis due to its simplicity and interpretability. David W. Hosmer et al. [3] (2013) discussed the effectiveness of Logistic Regression in binary classification problems, where the outcome variable has two possible classes. This algorithm estimates the probability of occurrence of an event, making it suitable for predicting whether an individual is likely to have autism or not. Logistic Regression performs well when the relationship between features and the target variable is approximately linear. Additionally, its interpretability allows healthcare professionals to understand the contribution of each feature in the prediction process.

Support Vector Machine (SVM), introduced by Vladimir Vapnik[4] (1995), is another powerful classification algorithm that has been extensively used in biomedical applications. SVM constructs an optimal hyperplane that separates data points into different classes with maximum margin, thereby improving classification accuracy. It is particularly effective in handling high-dimensional data and complex decision boundaries. In autism prediction studies, SVM has demonstrated strong performance due to its ability to manage nonlinear relationships through kernel functions, making it suitable for complex medical datasets.

Decision Tree algorithms are also widely used in classification problems due to their simplicity and ease of interpretation. These models represent decisions in a tree-like structure, making them highly intuitive for understanding the decision-making process. However, individual decision trees are prone to overfitting, especially when dealing with complex datasets. To overcome this limitation, Leo Breiman[5] (2001) introduced the Random Forest algorithm, which combines multiple decision trees to improve accuracy and generalization. Random Forest uses a bagging approach, where multiple subsets of data are used to train different trees, and the final prediction is obtained by aggregating their outputs. This ensemble method significantly reduces overfitting and improves model stability, making it highly effective for medical prediction tasks.

Boosting techniques have further enhanced the performance of machine learning models by focusing on improving weak learners. Yoav Freund and Robert Schapire[6]. (1997) introduced the AdaBoost algorithm, which iteratively adjusts the weights of misclassified instances to improve overall model accuracy. This approach enables the model to focus on difficult cases and enhances its predictive performance. AdaBoost has been successfully applied in various classification problems, including healthcare applications, due to its ability to create strong classifiers from weak learners.

Advanced gradient boosting algorithms have gained significant attention in recent years due to their superior performance and scalability. XGBoost, developed by Tianqi Chen (2016), incorporates regularization techniques to prevent overfitting and improve model generalization. It is designed for high efficiency and scalability, making it suitable for large datasets. XGBoost has been widely used in machine learning competitions and real-world applications due to its high predictive accuracy and robustness [7]. LightGBM, developed by Microsoft, further improves computational efficiency by using histogram-based learning and leaf-wise tree growth strategies. This makes it particularly suitable for large-scale datasets and real-time applications [8]. Similarly, CatBoost, developed by Yandex[9]., is specifically designed to handle categorical data effectively without extensive preprocessing. It reduces prediction bias and improves accuracy, especially in datasets with mixed data types.

Deep learning techniques have also been explored for autism prediction due to their ability to learn hierarchical representations of data. Yann LeCun, Yoshua Bengio, and Geoffrey Hinton (2015) highlighted the potential of deep neural networks in extracting complex features from raw data. Models such as Convolutional Neural Networks (CNN) and Recurrent Neural Networks (RNN) have been applied in analyzing behavioral data, speech patterns, and brain imaging data for autism detection. These models reduce the need for manual feature extraction and can automatically identify significant patterns, thereby improving prediction performance.

Feature engineering plays a vital role in improving the effectiveness of machine learning models. Pedro Domingos (2012) [10] emphasized that the success of a machine learning model largely depends on the quality of features used. In autism prediction, feature engineering involves extracting meaningful attributes such as behavioral scores,

communication patterns, and developmental indicators. Techniques such as feature transformation, normalization, and interaction feature generation help in enhancing model performance by providing more informative inputs to the learning algorithms .

Feature selection is another crucial step in handling high-dimensional data. Isabelle Guyon (2003) [11] discussed that feature selection techniques help in reducing dimensionality, improving model performance, and minimizing overfitting. Methods such as Chi-square test and SelectKBest are widely used to identify the most relevant features for classification tasks. By eliminating redundant and irrelevant features, these methods improve computational efficiency and enhance the generalization capability of machine learning models.

One of the major challenges in medical datasets is class imbalance, where the number of instances in one class significantly outweighs the other. Nitesh V. Chawla (2002) [12] explained that imbalanced datasets can lead to biased models that favor the majority class, resulting in poor detection of minority cases. In the context of autism prediction, this can lead to misclassification of affected individuals. Techniques such as oversampling, undersampling, and Synthetic Minority Over-sampling Technique (SMOTE) are commonly used to address this issue by balancing the dataset and improving model performance.

Evaluation of machine learning models is essential to ensure their reliability and effectiveness in real-world applications. Tom Fawcett[13] (2006) highlighted the importance of Receiver Operating Characteristic (ROC) curves in evaluating classification models. Metrics such as accuracy, precision, recall, F1-score, and Area Under the Curve (AUC) provide a comprehensive understanding of model performance. These evaluation measures are crucial in healthcare applications, where accurate prediction can significantly impact patient outcomes.

Recent studies have also explored hybrid approaches that combine multiple machine learning algorithms to improve prediction accuracy. Ensemble methods and hybrid models leverage the strengths of different algorithms to achieve better performance compared to individual models. Additionally, the integration of machine learning with real-time data processing systems has opened new possibilities for early and continuous monitoring of autism symptoms.

From the above literature, it is evident that machine learning techniques have significantly advanced the field of autism prediction by providing efficient and automated solutions for early diagnosis. The use of advanced algorithms, feature engineering, and data balancing techniques has improved prediction accuracy and reliability. However, challenges such as optimal feature selection, handling imbalanced datasets, computational complexity, and model optimization still remain. Therefore, this project aims to address these challenges by implementing effective feature engineering methods, SelectKBest feature selection, and multiple machine learning algorithms to develop a robust and efficient autism prediction system capable of delivering accurate and timely results.

CHAPTER 3

CHAPTER 3

SYSTEM SPECIFICATION

3.1 HARDWARE SPECIFICATION

- Processor : 12th Generation Intel(R) Core(TM) i5 / i7 Processor, 1.70 GHz or above
- RAM : 8 GB (Minimum) / 16 GB (Recommended)
- Monitor : LED Display
- Graphics : Integrated Graphics (Intel Iris Xe or equivalent)
- Storage : 256 GB SSD or above

3.2 SOFTWARE SPECIFICATION

- Operating System : Windows 10 / Windows 11
- Platform : Jupyter Notebook / Google Colab
- Language : Python 3.9
- Packages : NumPy, Pandas, Matplotlib, Seaborn, Scikit-learn, XGBoost, LightGBM,
- CatBoost, Imbalanced-learn

Algorithm Used :

- Logistic Regression
- Support Vector Machine (SVM)
- Random Forest
- Decision Tree
- AdaBoost
- XGBoost
- LightGBM
- CatBoost

3.3 SOFTWARE DESCRIPTION

3.3.1. Python

Python 3.9 is a powerful, versatile, and easy-to-learn programming language used by developers, data scientists, educators, and researchers alike. Released as part of the Python 3.x series, Python 3.9 introduces several new features, optimizations, and bug fixes that improve its performance, stability, and usability for building various types of applications.

History Of Python 3.9

Python 3.9 is part of the Python 3.x series, which brought significant changes to the language, focusing on readability, simplicity, and modern software development practices. Python 3.9 was officially released as the latest stable version in the Python 3 line after several years of development.

Development and Pre-release Versions

Python 3.9 development began after the release of Python 3.8, and it was part of the continuous improvement of the Python language under the guidance of the Python Software Foundation (PSF) and Python's BDFL (Benevolent Dictator for Life). Guido van Rossum, who retired from his leadership role in 2018, the major goals for Python 3.9 included improving performance, adding modern features, and cleaning up the language.

Release Candidate:

By August 2020, a release candidate (RC) version was available, which allowed users to test the near-final version of Python 3.9 with fewer bugs than the earlier beta releases.

Key Dates and Timeline

Python 3.9.0 Alpha 1 – May 2020

The first official alpha version for Python 3.9 was released, introducing the first set of planned features, such as the new zoneinfo module for IANA time zone support and improvements in string handling.

Python 3.9.0 Beta 1 – July 2020

The first beta release for Python 3.9 was released, which was intended for wider testing of the features and fixes.

Python 3.9.0 Release Candidate 1 – August 2020

The first release candidate allowed users to test Python 3.9 before the official stable release.

Python 3.9.0 Final Release – October 5, 2020

Python 3.9.0 was officially released as the stable version of Python, and this version came with several exciting features like dictionary merge operators, type hinting improvements, new string methods (`removeprefix()` and `removesuffix()`), and more.

Python 3.10: After Python 3.9, Python 3.10 was released in October 2021 and introduced even more advanced features, including pattern matching, and new syntax enhancements. Python 3.10 also continues the trend of making the language more efficient, user-friendly, and versatile.

Python 3.9's Legacy: Python 3.9 laid the foundation for some of the features and improvements found in later versions, and many of the features introduced in 3.9 (like dictionary operators, type hinting, and the `zoneinfo` module) have been widely adopted and are now considered best practices in Python development.

3.3.2. Features Of Python

- **Simple and Easy to Learn:**
 - Python 3.9 has a clean and simple syntax that is easy to understand, making it accessible for beginners and experienced developers alike. Its readability reduces the time needed to write and debug code.
- **Type Hinting Improvements:**
 - Python 3.9 introduced improvements to type hinting, allowing developers to use built-in collection types like `list`, `dict`, and `tuple` directly in type hints without importing from the `typing` module.

- **Dictionary Merge and Update Operators:**
 - Python 3.9 introduced two new operators for dictionaries: the merge operator (`|`) and the update operator (`|=`), making it easier to combine and update dictionaries in a clean and readable way.
- **New String Methods:**
 - Python 3.9 added two new string methods, `removeprefix()` and `removesuffix()`, which allow developers to easily remove prefixes and suffixes from strings without using complex slicing or conditional statements.
- **Improved Performance:**
 - Python 3.9 includes several performance improvements, including a faster and more efficient CPython interpreter, which speeds up the execution of Python code.
- **zone info Module:**
 - Python 3.9 introduced the `zoneinfo` module, which provides support for the IANA time zone database, making it easier to work with time zones in Python applications.
- **Open Source and Free:**
 - Python 3.9 is open-source and freely available, which means anyone can use, modify, and distribute it. This encourages a large and active community that contributes to its development and improvement.
- **Platform Independent:**
 - Python 3.9 can run on multiple platforms, including Windows, Linux, and macOS, without requiring any modifications to the code. This makes it a versatile and portable language.
- **Extensive Standard Library:**
 - Python 3.9 comes with a large standard library that includes modules for various tasks such as file handling, web development, data processing, and networking, reducing the need for external libraries.
- **Dynamic Typing:**
 - Python 3.9 uses dynamic typing, meaning variables do not need to be explicitly declared with a type. The interpreter automatically determines the type of a variable based on the assigned value.

3.3.3. Python Packages

NumPy

NumPy (Numerical Python) is a fundamental package for scientific computing in Python. It provides support for large, multi-dimensional arrays and matrices, along with a large collection of high-level mathematical functions to operate on these arrays. NumPy is widely used in data science, machine learning, and scientific research because of its efficiency and ease of use. It allows developers to perform complex numerical operations quickly, making it an essential tool for any data-driven application.

NumPy provides an array object called `ndarray`, which is faster and more efficient than Python's built-in `list`. It supports operations like element-wise computation, matrix multiplication, and linear algebra. NumPy also integrates well with other libraries like Pandas, Matplotlib, and Scikit-learn, making it a core component of the Python data science ecosystem. In this project, NumPy is used for array creation and manipulation, performing statistical computations, and supporting the data preprocessing pipeline.

Pandas

Pandas is an open-source data analysis and manipulation library for Python. It provides easy-to-use data structures such as `DataFrame` and `Series`, which allow developers to work with structured data efficiently. Pandas is particularly useful for loading, cleaning, transforming, and analyzing datasets, making it one of the most widely used libraries in data science and machine learning projects.

The `DataFrame` is a two-dimensional labeled data structure, similar to a table in a database or a spreadsheet, which allows for easy manipulation of rows and columns. Pandas supports a wide range of operations such as filtering, grouping, merging, and reshaping data. It also provides functions for handling missing values, reading data from various file formats like CSV and Excel, and performing time-series analysis. In this project, Pandas is used to load the autism screening dataset, perform data cleaning, handle missing values, and prepare the data for model training.

Matplotlib

Matplotlib is a comprehensive library for creating static, animated, and interactive visualizations in Python. It provides a wide range of plotting functions that allow developers to create charts, graphs, and other visual representations of data. Matplotlib is widely used in data science and machine learning for exploring datasets and presenting results in a visually appealing manner.

Matplotlib's pyplot module provides a MATLAB-like interface for creating plots and charts. It supports various types of visualizations such as line plots, bar charts, scatter plots, histograms, and pie charts. Matplotlib also supports customization of visual elements such as colors, fonts, labels, and gridlines. In this project, Matplotlib is used to visualize data distributions, plot model performance metrics, and generate confusion matrix charts to evaluate the performance of classification models.

Seaborn

Seaborn is a Python data visualization library based on Matplotlib. It provides a highlevel interface for drawing attractive and informative statistical graphics. Seaborn is particularly useful for visualizing complex datasets and relationships between variables. It includes built-in themes and color palettes that make it easier to create visually appealing charts without extensive customization.

Seaborn provides functions for creating various types of plots such as heatmaps, violin plots, pair plots, and box plots. It integrates well with Pandas DataFrames, allowing developers to directly plot data from a DataFrame without additional data transformation. In this project, Seaborn is used to create correlation heatmaps to identify relationships between features, distribution plots to understand the spread of data, and count plots to visualize the class distribution of the autism dataset.

Scikit-learn

Scikit-learn is a free and open-source machine learning library for Python. It provides simple and efficient tools for data mining and data analysis, built on top of NumPy, SciPy, and Matplotlib. Scikit-learn includes a wide range of machine learning algorithms for

classification, regression, clustering, and dimensionality reduction, making it one of the most widely used libraries in machine learning projects.

Scikit-learn follows a consistent API design, where all machine learning algorithms implement `fit()`, `predict()`, and `score()` methods, making it easy to experiment with different models. It also provides utilities for model evaluation, cross-validation, feature selection, and hyperparameter tuning. In this project, Scikit-learn is used to implement multiple classification algorithms including Logistic Regression, Support Vector Machine, Random Forest, Decision Tree, and AdaBoost. It is also used for splitting the dataset into training and testing sets, scaling features, and evaluating model performance using metrics such as accuracy, precision, recall, and F1-score.

XGBoost

XGBoost (Extreme Gradient Boosting) is an optimized and scalable gradient boosting library designed for speed and performance. It was developed by Tianqi Chen and has become one of the most popular machine learning algorithms, especially for structured and tabular data. XGBoost uses a gradient boosting framework, which builds an ensemble of decision trees in a sequential manner, where each new tree corrects the errors made by the previous ones.

XGBoost includes several regularization techniques such as L1 and L2 regularization to prevent overfitting and improve generalization. It also supports parallel processing, which significantly speeds up the model training process. XGBoost has achieved top performance in many machine learning competitions and is widely used in industry for classification and regression tasks. In this project, XGBoost is used as one of the primary classification algorithms for autism prediction, taking advantage of its high accuracy and ability to handle complex patterns in the data.

LightGBM

LightGBM (Light Gradient Boosting Machine) is a gradient boosting framework developed by Microsoft that uses tree-based learning algorithms. It is designed to be highly efficient and scalable, making it suitable for large datasets. LightGBM uses a leaf-wise tree growth strategy, which grows the tree by choosing the leaf with the highest gain, resulting in

faster training and better accuracy compared to level-wise tree growth strategies used by other gradient boosting algorithms.

LightGBM introduces two key techniques: Gradient-based One-Side Sampling (GOSS) and Exclusive Feature Bundling (EFB), which significantly reduce the computational cost while maintaining accuracy. It supports categorical features natively and handles missing values automatically. In this project, LightGBM is used as a classification algorithm for autism prediction due to its fast training speed and high performance on tabular datasets.

CatBoost

CatBoost (Categorical Boosting) is a gradient boosting algorithm developed by Yandex that is specifically designed to handle categorical features efficiently. Unlike other gradient boosting algorithms that require manual encoding of categorical features, CatBoost can work directly with categorical data without any preprocessing, making it easy to use and highly efficient.

CatBoost uses ordered boosting and a novel algorithm for handling categorical features called ordered target statistics, which reduces overfitting and improves model accuracy. It also supports GPU training, which significantly speeds up the training process for large datasets.

CatBoost is known for its robustness and ease of use, as it requires minimal hyperparameter tuning to achieve good performance. In this project, CatBoost is used to classify autism screening data, taking advantage of its ability to handle categorical variables such as ethnicity, country of residence, and relation without requiring explicit encoding.

Imbalanced-learn

Imbalanced-learn is a Python library that provides tools for dealing with imbalanced datasets in machine learning. Imbalanced datasets occur when one class has significantly more samples than the other, which can cause machine learning models to be biased toward the majority class, resulting in poor performance on the minority class. This is a common problem in medical and healthcare datasets, where the number of positive cases is often much smaller than the number of negative cases.

Imbalanced-learn provides various resampling techniques to address class imbalance, including oversampling methods such as SMOTE (Synthetic Minority Over-sampling Technique), undersampling methods such as Random Under Sampler, and combination methods that use both oversampling and undersampling. SMOTE generates synthetic samples for the minority class by interpolating between existing samples, which helps to balance the dataset and improve the performance of the classification models. In this project, Imbalancedlearn is used to apply SMOTE to the autism screening dataset to address the class imbalance problem and improve the sensitivity and accuracy of the prediction models.

Jupyter Notebook

Jupyter Notebook is an open-source web application that allows you to create and share documents containing live code, equations, visualizations, narrative text, and other multimedia resources. It is widely used in data science, machine learning, scientific research, and education for interactive computing. "Jupyter" stands for Julia, Python, and R, but it now supports many programming languages. The notebooks are a mix of code (usually Python), Markdown, and outputs like graphs, tables, and more.

Features of Jupyter Notebook

- Interactive Coding:
 - You can write and execute code interactively in the notebook, which allows for real-time testing and iteration.
- Markdown Support:
 - Markdown can be used to write rich text, including formatted text, headers, bullet points, images, and even LaTeX for mathematical equations.
- Visualization:
 - Jupyter Notebook supports embedding visualizations from libraries like Matplotlib, Seaborn, Plotly, Bokeh, and Altair directly into the notebook.
- Rich Media Support:
 - It can include images, videos, and even interactive widgets within your notebooks to enhance presentations or explanations.
- Kernel-Based Execution:

- A Jupyter Notebook connects to a kernel (a computing engine that runs the code), which is language-specific (e.g., Python, R, Julia). You can switch kernels depending on the language you're using.
- Reproducibility:
 - The ability to run code in individual cells allows for reproducible workflows, which is essential in data science and research.
- Modular Code Execution:
 - The code is divided into cells, and you can run each cell independently, making it easier to debug and modify specific parts of your code.

Running Jupyter Notebook:

- Once started, Jupyter opens in a browser window. The main page allows you to create new notebooks, open existing notebooks, and manage files.
- To execute a code cell, simply select the cell and press Shift + Enter.
- You can add new cells by clicking the + button in the toolbar or by pressing B (below) or A (above) in command mode.
- To save your notebook, press Ctrl + S or use the File menu and select Save.
- Notebooks can be exported to various formats including HTML, PDF, and Python script using the File > Download As option.

3.3.4 Machine Learning Algorithms Used

1. Logistic Regression

Logistic Regression is a supervised machine learning algorithm primarily used for binary classification tasks. Despite its name, it is a classification technique that models the probability of a given input belonging to a particular class. It employs the logistic (sigmoid) function to map real-valued inputs into a range between 0 and 1, enabling probabilistic interpretation.

The sigmoid function is mathematically defined as:

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

where z represents the linear combination of input features:

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

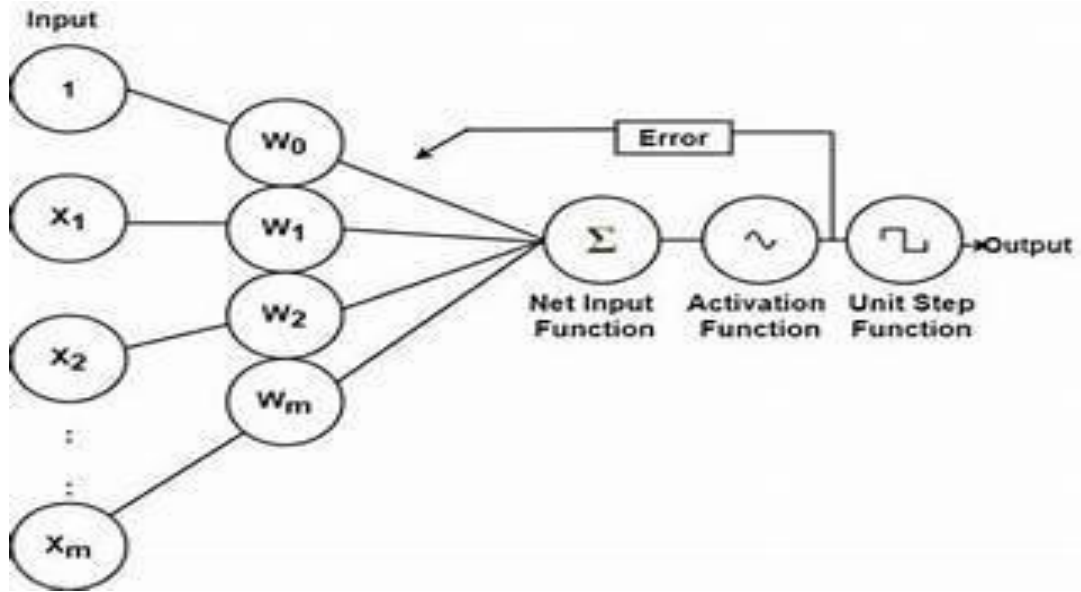


Figure 3.1 Logistic Regression

A threshold value, typically 0.5, is used to assign class labels, where outputs greater than the threshold are classified as the positive class, and those below it as the negative class.

Logistic Regression estimates the probability of class membership using the following decision function:

$$P(Y = 1 | X) = \frac{1}{1 + e^{-z}}$$

The final predicted class is determined based on the value of the probability function. Logistic Regression is computationally efficient and highly interpretable, making it suitable as a baseline model. It performs well when the relationship between features and the target variable is approximately linear. In this study, Logistic Regression is utilized as a baseline classifier for autism prediction, and its performance is compared with more advanced models.

2. Support Vector Machine (SVM)

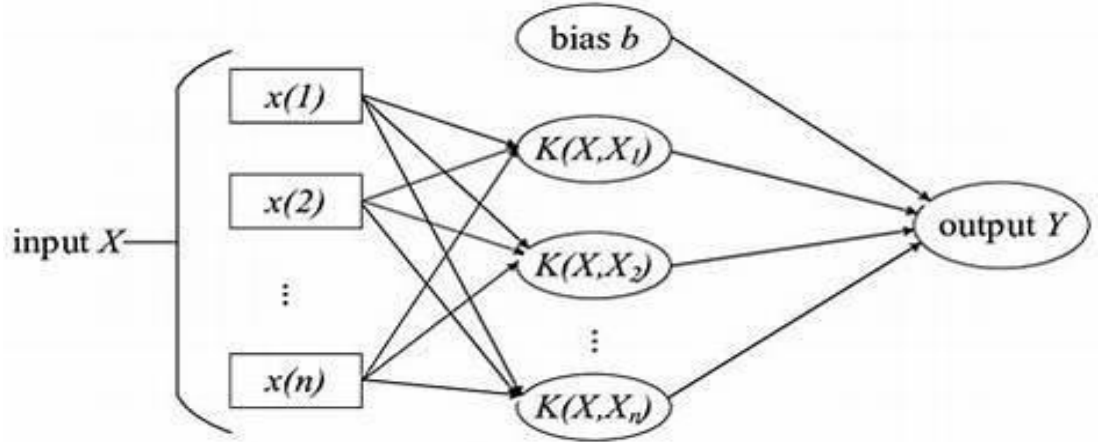


Figure3.2 Support Vector Machine(SVM)

The figure represents the kernel-based architecture of the Support Vector Machine (SVM). The input vector $X = [x_1, x_2, \dots, x_n]$ is mapped into a higher-dimensional feature space using kernel functions $K(X, X_i)$, which compute similarity between the input and support vectors. A bias term b is also included to shift the decision boundary for optimal separation.

The final decision function is computed as

$$f(X) = \sum_{i=1}^n \alpha_i y_i K(X, X_i) + b$$

where α_i are Lagrange multipliers and y_i are class labels. Only selected training samples, known as support vectors, contribute to the decision function. The sign of $f(X)$ determines the predicted class label. This formulation enables SVM to effectively handle both linear and non-linear classification problems with high accuracy and strong generalization.

3. Random Forest

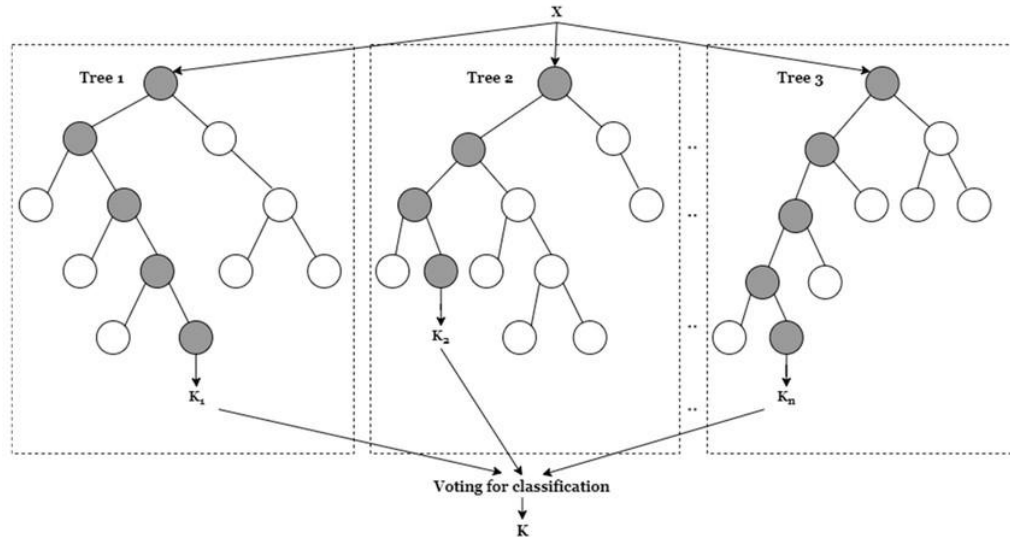


Figure 3.3 Random Forest

The figure illustrates the working principle of the Random Forest algorithm, where a single input instance X is passed simultaneously into multiple independently constructed decision trees. Each tree is trained on a different subset of the training data using the bootstrap sampling (bagging) technique, ensuring diversity among the trees. In addition, at each split within a tree, a random subset of features is selected, which further reduces correlation between individual trees and improves generalization.

Given an input instance X , it is passed through multiple decision trees:

$$X \rightarrow \{T_1, T_2, \dots, T_n\}$$

Each tree produces an individual prediction:

$$\{K_1, K_2, \dots, K_n\}$$

The final output is obtained using a majority voting mechanism:

$$K = \text{mode}(K_1, K_2, \dots, K_n)$$

Within each tree, the input X traverses from the root node to a leaf node based on learned decision rules, producing individual predictions $K_1, K_2, \dots, K_n$. These outputs are then aggregated in the final stage using a majority voting mechanism for classification. The final predicted class K corresponds to the class receiving the highest number of votes. This

ensemble approach enhances prediction accuracy, reduces overfitting, and provides robust performance compared to a single decision tree.

4. Decision Tree

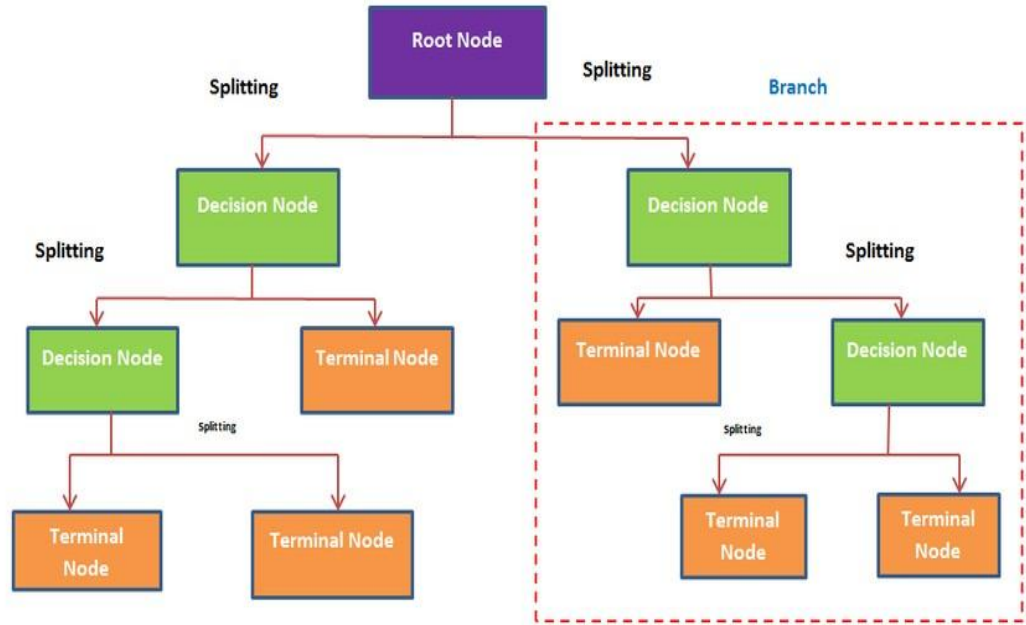


Figure 3.4 Decision Tree

Decision Tree is a non-parametric supervised learning algorithm used for both classification and regression tasks. It represents decisions in a hierarchical tree-like structure consisting of nodes and branches, where each internal node denotes a feature-based test, each branch represents the outcome of the test, and each leaf node corresponds to a final prediction.

Given an input feature vector:

$$X = [x_1, x_2, \dots, x_n]$$

the model recursively partitions the dataset based on a selected feature and threshold value.

The splitting criterion is determined using measures such as **Gini Index** or **Entropy (Information Gain)**.

The entropy is defined as:

$$H(S) = - \sum_{i=1}^c p_i \log_2 p_i$$

where p_i represents the probability of class i . The information gain for a split is computed as:

$$IG(S, A) = H(S) - \sum_{v \in \text{Values}(A)} \frac{|S_v|}{|S|} H(S_v)$$

where

- S is the dataset
- A is the feature
- S_v is the subset after the split

The final prediction is obtained by traversing the tree from the root node to a leaf node based on feature values.. In this study, Decision Tree is used both as an individual classifier and as a base learner for ensemble methods such as AdaBoost.

5. AdaBoost

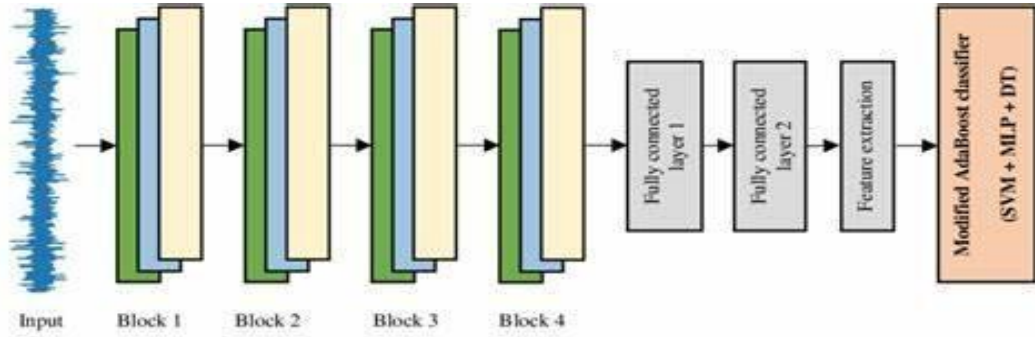


Figure 3.5 AdaBoost

AdaBoost (Adaptive Boosting) is an ensemble learning algorithm that combines multiple weak classifiers to form a strong classifier. It operates sequentially, where each subsequent model focuses more on the samples that were misclassified by previous models.

Given a training dataset:

$$D = \{(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)\}$$

AdaBoost assigns equal weights to all samples initially:

$$w_i = \frac{1}{n}$$

At each iteration t , a weak classifier $h_t(x)$ is trained, and its error is computed as:

$$\epsilon_t = \sum_{i=1}^n w_i \cdot I(y_i \neq h_t(x_i))$$

where $I(\cdot)$ is the indicator function.

The classifier weight is then calculated as:

$$\alpha_t = \frac{1}{2} \ln \left(\frac{1 - \epsilon_t}{\epsilon_t} \right)$$

The sample weights are updated as:

$$w_i \leftarrow w_i \cdot e^{-\alpha_t y_i h_t(x_i)}$$

and normalized so that $\sum w_i = 1$.

The final strong classifier is given by:

$$H(x) = \text{sign} \left(\sum_{t=1}^T \alpha_t h_t(x) \right)$$

In this study, **Decision Stumps** (Decision Trees with depth = 1) are used as weak learners. AdaBoost improves classification performance by focusing on difficult samples and reducing both bias and variance, making it highly effective for binary classification problems such as autism prediction.

CHAPTER 4

CHAPTER 4

4. METHODOLOGY AND IMPLEMENTATION

4.1 SYSTEM ARCHITECTURE

System architecture represents the overall structure of the proposed system, defining its components and their interactions to achieve accurate ASD prediction. The process begins with the input dataset, followed by preprocessing to clean and organize the data. Missing values are handled to ensure data consistency, and feature engineering is applied to extract meaningful information. Feature scaling and selection are then performed to improve model performance and reduce complexity. The dataset is split into training and testing sets, where machine learning algorithms are trained and evaluated. Finally, performance metrics and accuracybased evaluation are used to generate the final ASD prediction result



Figure 4.1 System Architecture

4.2 DATASET

A dataset is a collection of data, usually organized in a structured way, that is used for analysis, training machine learning models, or performing various types of research or experiments. It typically consists of a set of data points or records, each containing multiple features or attributes.

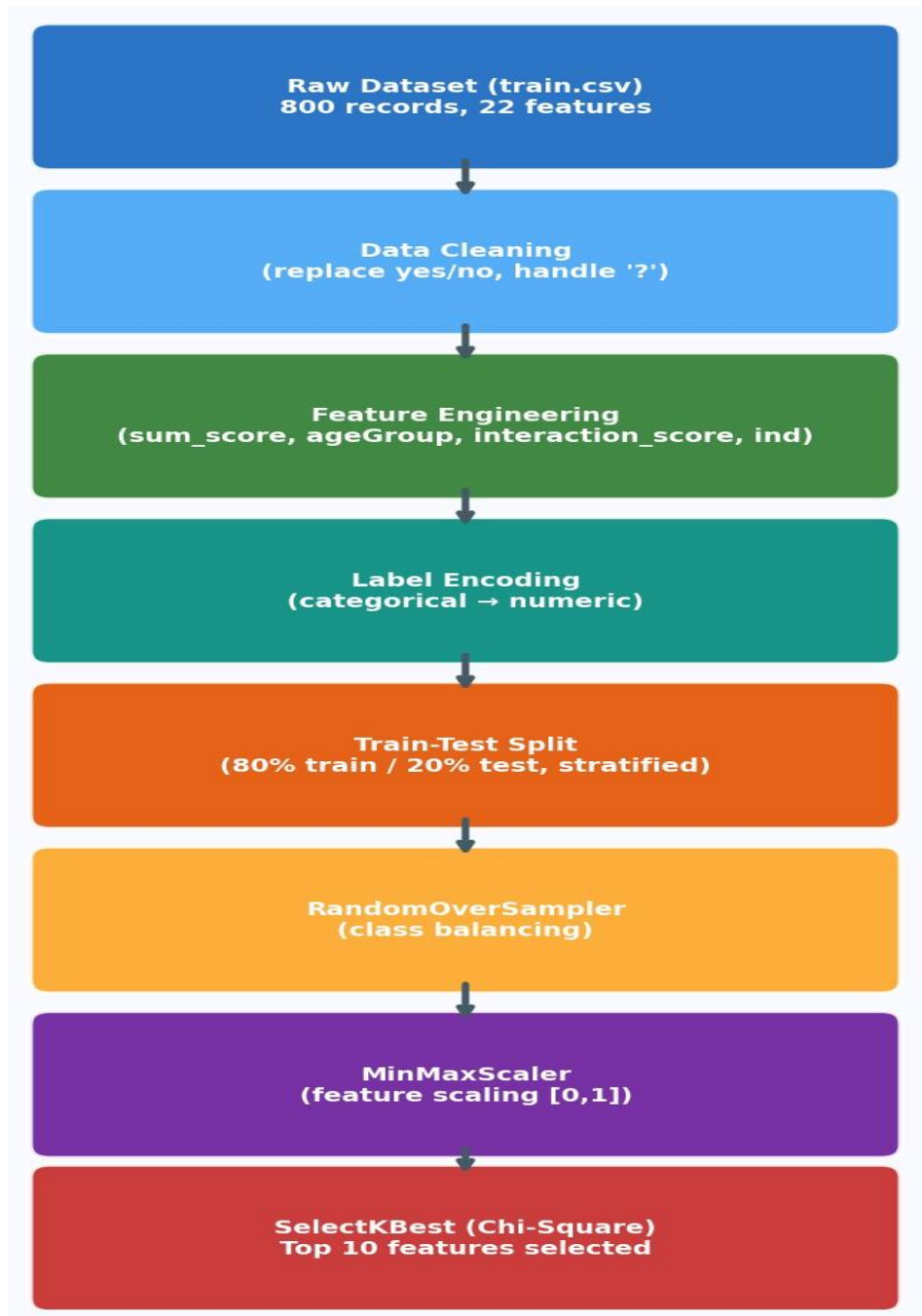


Figure 4.2 ML Preprocessing Pipeline Flowcart

The dataset used in this project is the Autism Screening Dataset collected from Kaggle. The train.csv file contains 800 records and 22 attributes including behavioral questionnaire responses (A1_Score to A10_Score), demographic information, and medical history. From the dataset, 640 records are selected for Training and 160 for Testing.

Table 4.1: Dataset Feature Description

Feature	Type	Description
A1_Score – A10_Score	Binary (0/1)	AQ-10 behavioral questionnaire responses
age	Continuous	Age of the individual in years
gender	Categorical	Biological sex (Male/Female)
ethnicity	Categorical	Ethnic background
jaundice	Binary	History of neonatal jaundice (yes/no)
austim	Binary	Family member with ASD diagnosis (yes/no)
country_of_res	Categorical	Country of residence
used_app_before	Binary	Prior use of screening app (yes/no)
result	Integer	AQ-10 questionnaire total score
relation	Categorical	Respondent's relation to subject
Class/ASD	Binary (0/1)	TARGET: ASD diagnosis

4.3 METHODOLOGY

Methodology refers to the systematic approach or process used to carry out a research or project. It outlines the methods and procedures used to collect, analyze, and interpret data in order to achieve the project's objectives. The ASD prediction system can be detected and classified using a comprehensive machine learning preprocessing and training pipeline.

4.3.1 DATA COLLECTION

The Autism Screening Dataset (train.csv) from Kaggle includes AQ-10 scores, demographic features, and a binary ASD target variable. The AQ-10 is a 10-question screening tool where Yes/No responses are converted into numerical values (1/0) for machine learning..

Table 4.2: AQ-10 Questionnaire Item

Question No	AQ-10 Questionnaire Item	Response Type	Description
A1	I often notice small sounds when others do not	Yes / No	Measures sensory sensitivity
A2	I usually concentrate more on the whole picture rather than small details	Yes / No	Evaluates attention style
A3	I find it easy to do more than one thing at once	Yes / No	Assesses multitasking ability
A4	If there is an interruption, I can switch back quickly	Yes / No	Measures cognitive flexibility
A5	I find it easy to ‘read between the lines’ in conversation	Yes / No	Evaluates social understanding
A6	I can tell if someone listening to me is getting bored	Yes / No	Measures social awareness
A7	I find it difficult to understand characters’ intentions in stories	Yes / No	Assesses theory of mind
A8	I like collecting information about categories of things	Yes / No	Measures patternbased interest
A9	I can understand emotions from facial expressions	Yes / No	Evaluates emotional recognition
A10	I find it difficult to understand people’s intentions	Yes / No	Measures social cognition

Each response contributes to the overall AQ-10 score, which is a strong indicator of autism-related behavioral traits. In this project, these responses are represented as features (A1_Score to A10_Score), forming the foundation for predictive modeling.

A1	A2	A3	A4	A5	A6	A7	A8	A9	A10	age	gender	ethnicity	jaundice	austim	result	Class/ASD
1	0	1	1	0	0	1	1	0	0	25	Male	White	no	no	6	1
0	0	0	1	0	0	1	0	1	0	17	Female	Asian	no	yes	3	0
1	1	0	1	1	0	1	0	1	1	34	Male	Others	no	no	7	1
0	1	1	0	0	1	0	1	0	0	12	Female	White	yes	no	4	0
1	0	1	1	1	0	1	1	0	1	8	Male	Asian	no	yes	7	1

Figure 4.3.1 Input Dataset

4.3.2 DATA PREPROCESSING

Data preprocessing is a fundamental and indispensable step in the machine learning workflow, as it significantly impacts both the performance and predictive accuracy of the resulting model. Raw datasets collected from real-world sources often contain inconsistencies, missing values, and irrelevant information, which can negatively affect model training if left unprocessed. Therefore, preprocessing ensures that the data is clean, consistent, and structured in a way that machine learning algorithms can interpret effectively. This step not only improves model efficiency but also helps in reducing noise, preventing biases, and facilitating faster convergence during training.

In this study, several preprocessing techniques were applied to the dataset to prepare it for model training. Binary string values such as "yes" and "no" were standardized to numerical form (1 and 0, respectively) to enable mathematical computations by the algorithms. Missing or ambiguous entries, represented by "?" or "others", were replaced with a consistent label "Others" to maintain uniformity. Non-predictive columns, including unique identifiers (ID) and descriptive age labels (age_desc), were removed to prevent irrelevant information from influencing the model. Finally, Label Encoding was applied to all remaining categorical features, converting them into numeric form so that the machine learning algorithms could process them efficiently.

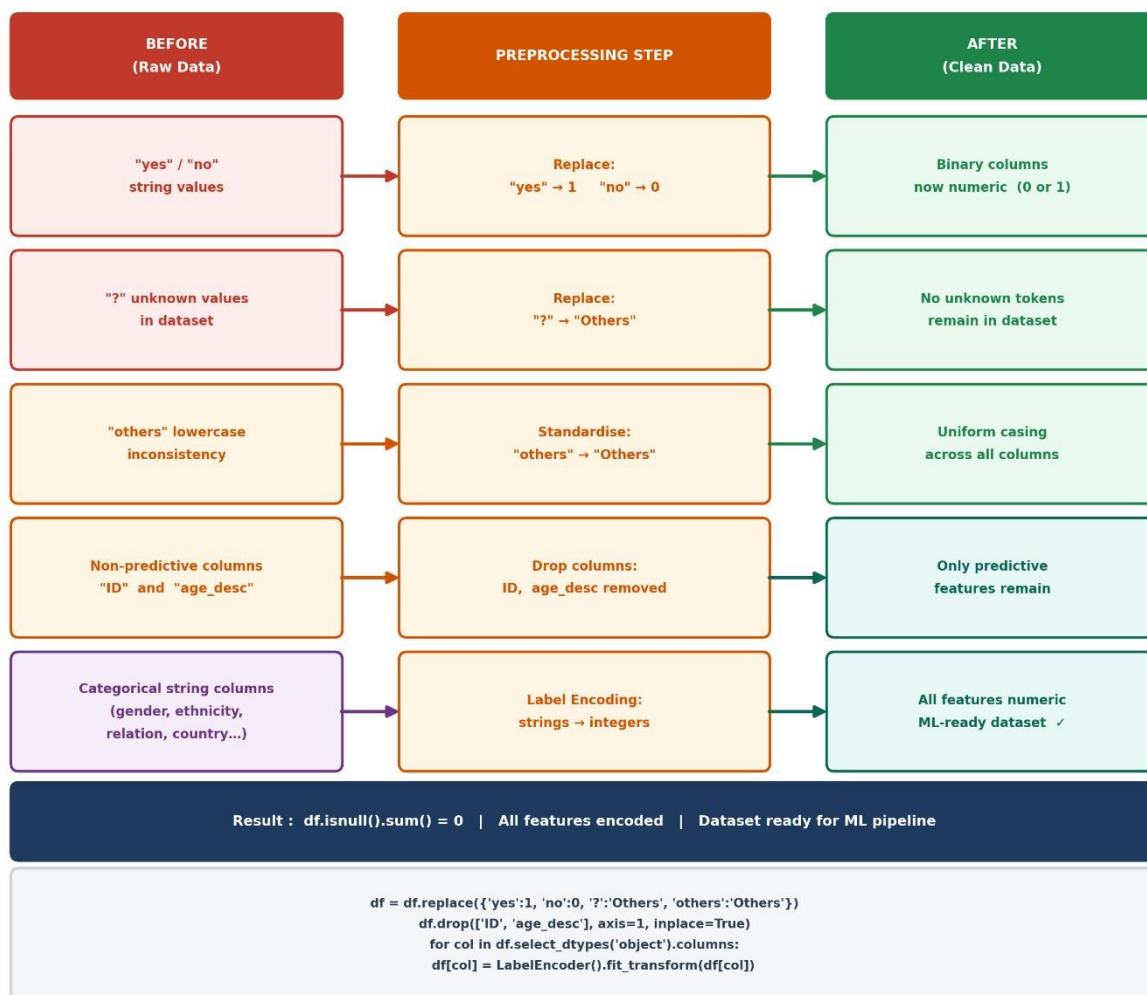


Figure 4.3.2 Handling Missing Values

4.3.3 FEATURE ENGINEERING

Feature engineering is the process of using domain knowledge to transform raw data into informative features that improve model performance. Three novel engineered features are introduced in this study:

sum_score: Computed as the sum of all ten AQ-10 behavioral response items (A1_Score + A2_Score + ... + A10_Score). This replicates the validated AQ-10 total screening score and is the single most predictive feature in the dataset.

ageGroup: Continuous age values are categorized into five clinically meaningful groups: Toddler (age < 4), Kid (4 ≤ age < 12), Teen (12 ≤ age < 18), Young (18 ≤ age < 40),

and Senior (age ≥ 40). This captures non-linear age-related patterns in ASD behavioral presentation.

interaction_score: A multiplicative interaction feature computed as $\text{sum_score} \times \text{austim}$. This captures the combined effect of high behavioral risk and family genetic predisposition for ASD. ind: A composite background risk indicator computed as $\text{austim} + \text{used_app_before} + \text{jaundice}$, ranging from 0 to 3.

4.3.4 SPLITTING DATA

To use the technique of train-test split to divide the dataset into two parts: a training set (used to train the ML models) and a testing set (used to evaluate the performance of the model). The training set is 80% of the total data (640 samples), while the testing set makes up the remaining 20% (160 samples). Stratified splitting ensures that the class distribution is preserved in both partitions. The split ensures that the model is tested on unseen data to evaluate its generalization ability. A fixed $\text{random_state} = 42$ is used for reproducibility.

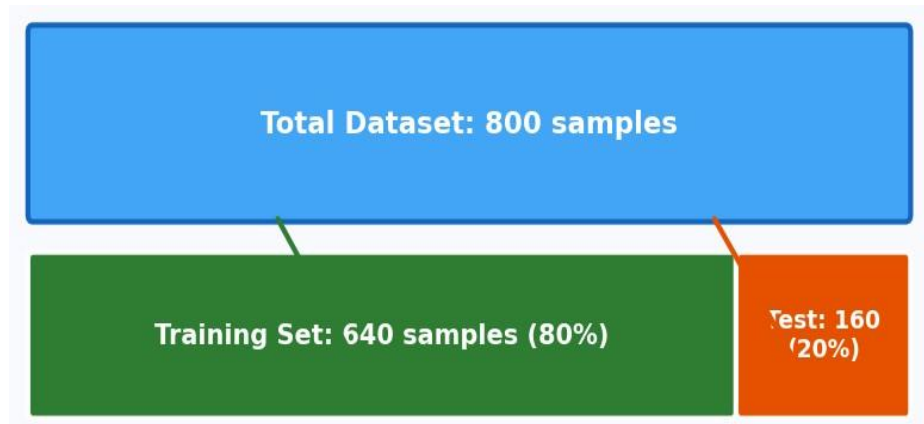


Figure 4.3.3 Training and Testing Split

4.3.5 ACCURACY BASED RESULT

After splitting the dataset into training and testing sets, training the machine learning models, and evaluating them, one of the most important metrics to assess the performance of the model is accuracy. However, to obtain a comprehensive view of model performance, additional metrics including precision, recall, F1-score, and AUC-ROC are also computed.

4.4 EVALUATION OF MACHINE LEARNING MODELS

4.4.1 LOGISTIC REGRESSION ALGORITHM

Logistic Regression is a statistical model commonly used for binary classification problems, where the goal is to predict one of two possible outcomes. Despite its name, logistic regression is used for classification, not regression. It is one of the simplest and most widely used algorithms for predicting categorical outcomes.

The core of logistic regression is the sigmoid function:

$$f(x) = 1 / (1 + e^{-x})$$

Where x is the weighted sum of input features and e is the base of the natural logarithm. The function maps any real-valued number to a value between 0 and 1, allowing interpretation as a probability.

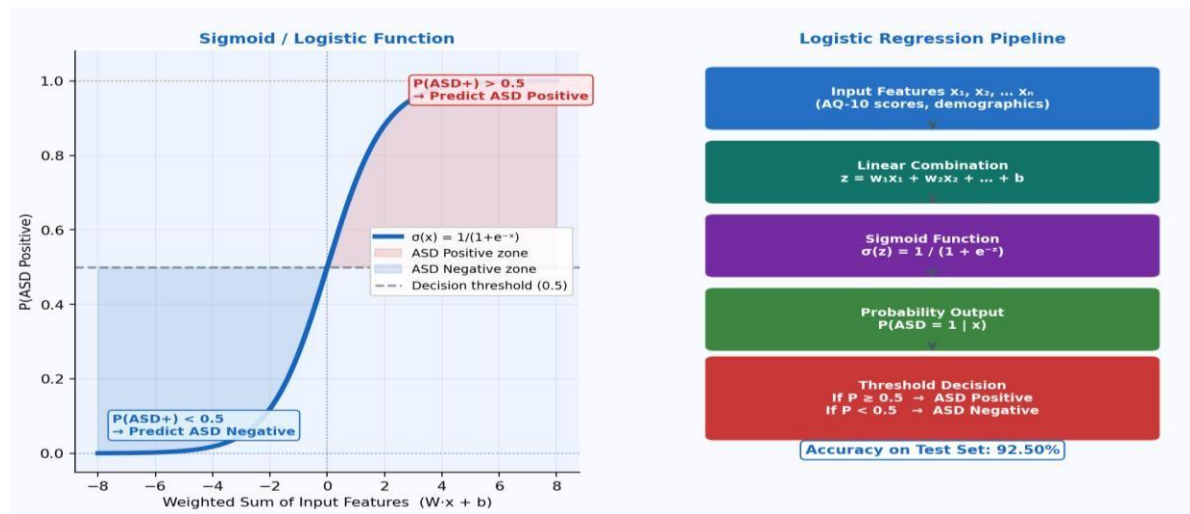


Figure 4.4.1 Logistic Regression Sigmoid Function

4.4.2 SUPPORT VECTOR MACHINE ALGORITHM

A Support Vector Machine (SVM) is a powerful supervised machine learning algorithm used primarily for classification, although it can also be used for regression tasks. SVMs are particularly effective in high-dimensional spaces and are widely used for classification tasks in both binary and multi-class settings.

The main idea behind SVM is to find a hyperplane that best separates the data into different classes. This hyperplane should not only separate the classes but also maximize the margin between the classes and the hyperplane itself. The equation of the hyperplane is:

$$\mathbf{W} \cdot \mathbf{x} + b = 0$$

Where w is the normal vector to the hyperplane, b is the bias term, and x represents the feature vector of a data point.

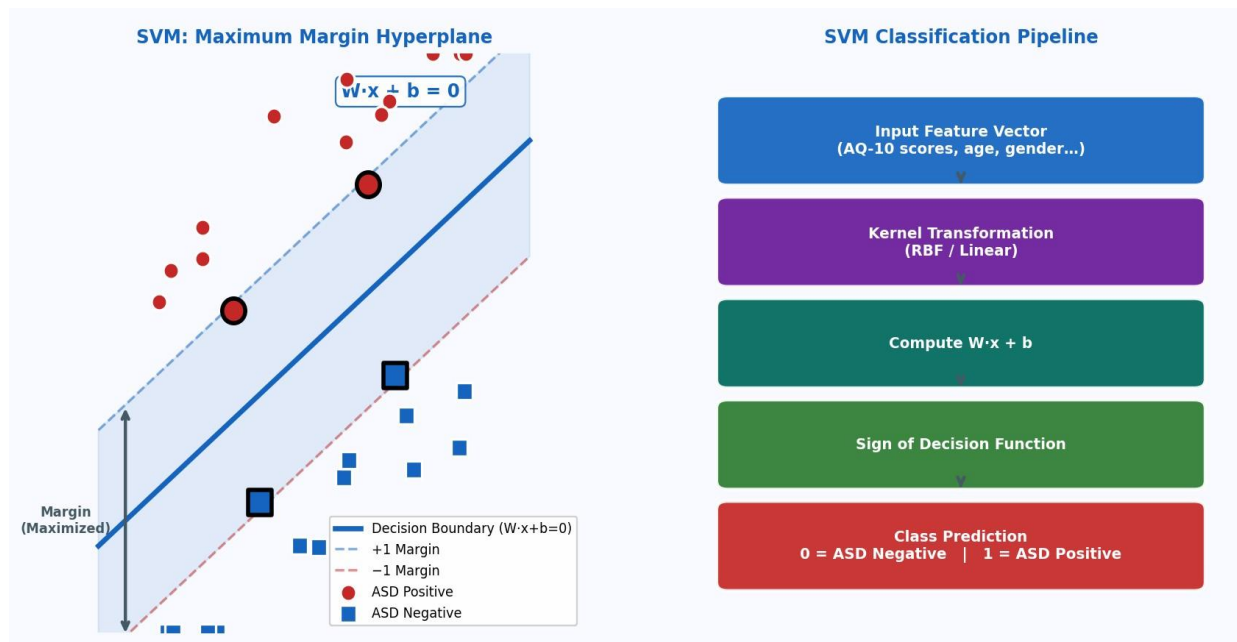


Figure 4.4.2 Support Vector Machine Algorithm

4.4.3 DECISION TREE ALGORITHM

Decision Tree is a simple yet effective algorithm that splits data based on feature values using information gain or Gini impurity as the splitting criterion. The tree structure makes it highly interpretable, allowing the decision path to be directly read as a set of if-then clinical rules. However, Decision Trees are prone to overfitting if not properly constrained through depth limiting or pruning.

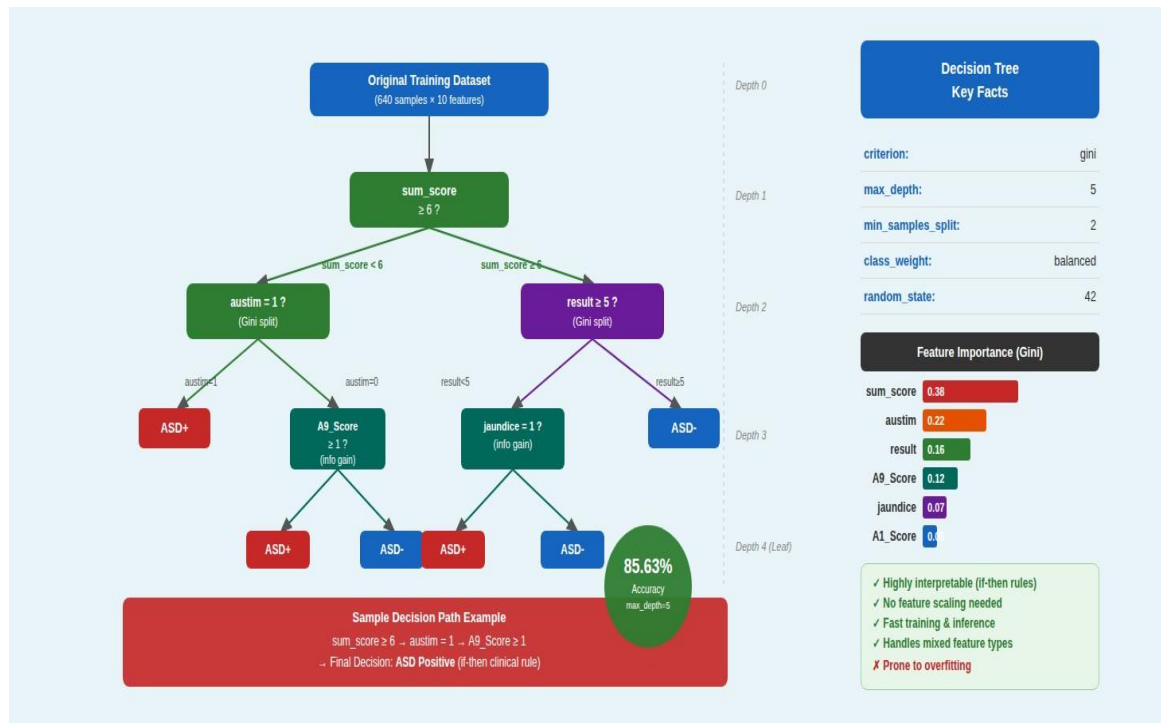


Figure 4.4.3 Decision Tree Algorithm

4.4.4 RANDOM FOREST ALGORITHM

Random Forest builds a large number of decision trees using a technique called bagging (bootstrap aggregating), where each tree is trained on a randomly sampled subset of the data with replacement. Additionally, at each split in a tree, only a random subset of features is considered, which introduces further diversity among the trees. This randomness ensures that the individual trees are less correlated with each other, improving the overall generalization ability of the model. During prediction, each tree independently produces an output, and the final result is determined by majority voting (for classification) or averaging (for regression). This approach significantly reduces variance and helps prevent overfitting, which is common in single decision trees.

One of the major strengths of Random Forest is its robustness and ability to handle high-dimensional data with minimal preprocessing. It can manage missing values, maintain accuracy even when a large proportion of data is missing, and is less affected by noise compared to boosting methods. The algorithm also provides feature importance measures, helping identify which variables contribute most to the prediction, making it useful for feature

selection. However, Random Forest models can be computationally expensive with large datasets and many trees, and they may lack interpretability compared to simpler models. Despite this, they are widely used in applications such as healthcare prediction, finance, and classification problems due to their strong performance and reliability

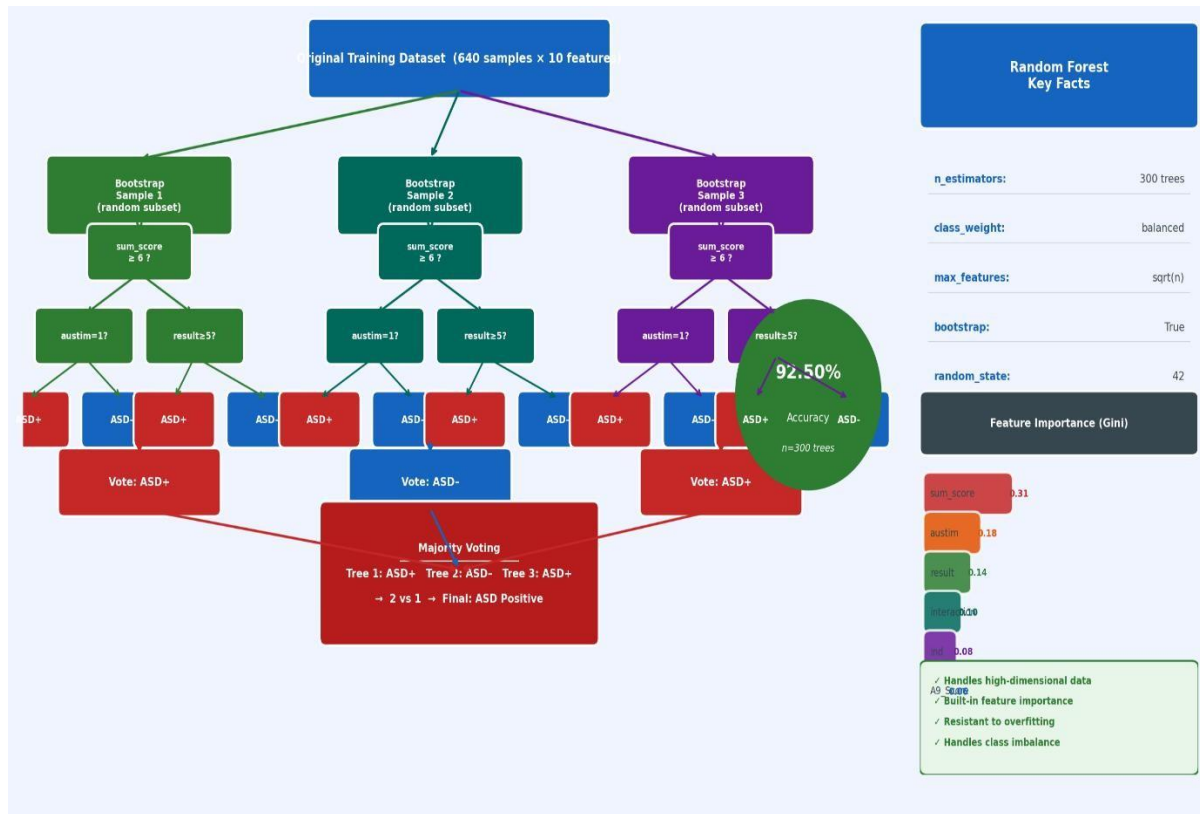


Figure 4.4.4 Random Forest Algorithm

4.4.5 ADABOOST ALGORITHM

AdaBoost works by sequentially adding weak learners, typically decision stumps (onelevel decision trees), where each new model focuses more on the mistakes made by the previous ones. Initially, all training samples are assigned equal weights, but after each iteration, the weights of misclassified samples are increased while correctly classified ones are decreased. This forces subsequent learners to pay more attention to hard-to-classify instances. Each weak learner is assigned a weight based on its accuracy, calculated using an error rate; learners with lower error receive higher influence in the final model. The algorithm minimizes

an exponential loss function, which strongly penalizes misclassifications, making it sensitive to noisy data and outliers.

One key advantage of AdaBoost is that it often achieves high accuracy without requiring complex models, and it reduces both bias and variance when properly tuned. However, it can be sensitive to noise because difficult or mislabeled data points may receive excessively high weights, leading to overfitting. AdaBoost is commonly used in classification tasks such as face detection, text classification, and medical diagnosis. It also has variants like AdaBoost.M1 (for binary classification) and AdaBoost.M2 (for multi-class problems). Additionally, it does not require extensive parameter tuning compared to other ensemble methods, making it relatively simple to implement while still being powerful in practice.



Figure 4.4.5 Adaboost Algorithm

4.4.6 XGBOOST ALGORITHM

XGBoost (eXtreme Gradient Boosting) is an optimized gradient boosting framework that incorporates L1 and L2 regularization, column and row subsampling, and efficient handling of sparse features. The regularized objective function is:

$$L(\Phi) = \sum_i l(y_i, \hat{y}_i) + \sum_k \Omega(f_k)$$

Where l is the cross-entropy loss and $\Omega(f) = \gamma T + \frac{1}{2}\lambda \|w\|^2$ penalizes tree complexity. XGBoost has consistently achieved state-of-the-art performance on structured tabular data classification tasks.

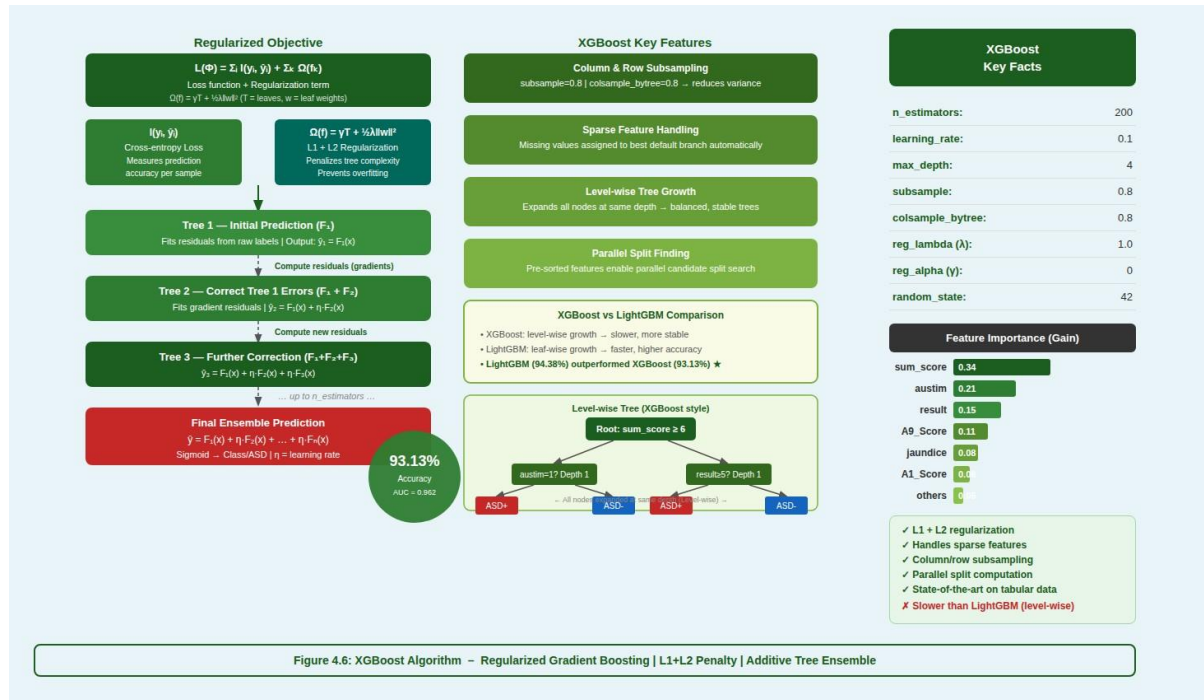


Figure 4.4.6 Xgboost Algorithm

4.4.7 LIGHTGBM ALGORITHM

LightGBM (Light Gradient Boosting Machine) is a highly efficient gradient boosting framework specifically designed to handle large-scale data with faster training speed and lower memory consumption, while still delivering high predictive performance. Unlike traditional boosting algorithms, LightGBM adopts a histogram-based technique that converts continuous feature values into discrete bins. This significantly reduces computational complexity and speeds up the training process, especially for large datasets. It also supports parallel and GPU-based learning, making it suitable for real-world applications where performance and scalability are critical.

A distinctive feature of LightGBM is its leaf-wise (best-first) tree growth strategy, where the algorithm expands the leaf node that yields the highest reduction in loss, rather than

growing trees level by level. This results in deeper and more optimized trees, improving model accuracy with fewer iterations. However, since this approach may lead to overfitting, parameters such as maximum depth, minimum data in leaf, and regularization techniques are used to control model complexity. LightGBM further enhances efficiency through Gradientbased One-Side Sampling (GOSS), which retains data points with large gradients (more informative samples) while randomly sampling those with small gradients, and Exclusive Feature Bundling (EFB), which combines mutually exclusive features to reduce dimensionality.

Due to these advanced optimizations, LightGBM performs exceptionally well on structured tabular data, such as the Autism Screening Dataset, where it achieved superior accuracy and AUC scores by effectively capturing complex patterns and relationships among behavioral and demographic features..

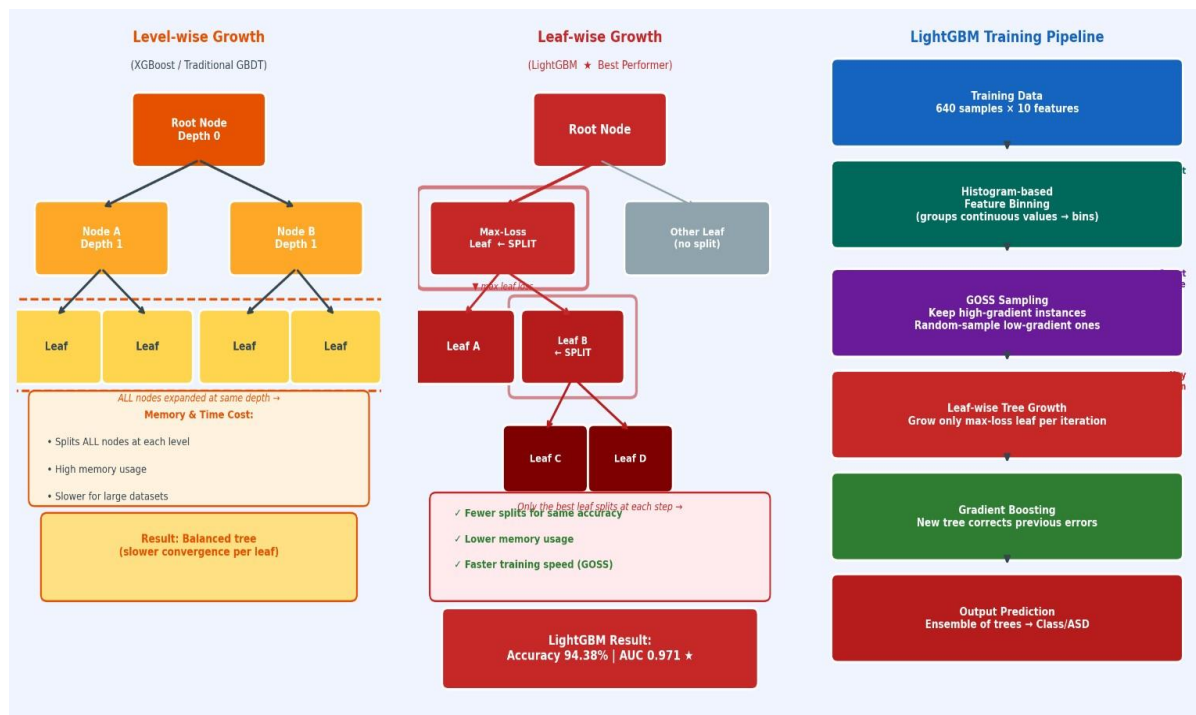


Figure 4.4.7 LightGBM Leaf-wise Growth

4.4.8 CATBOOST ALGORITHM

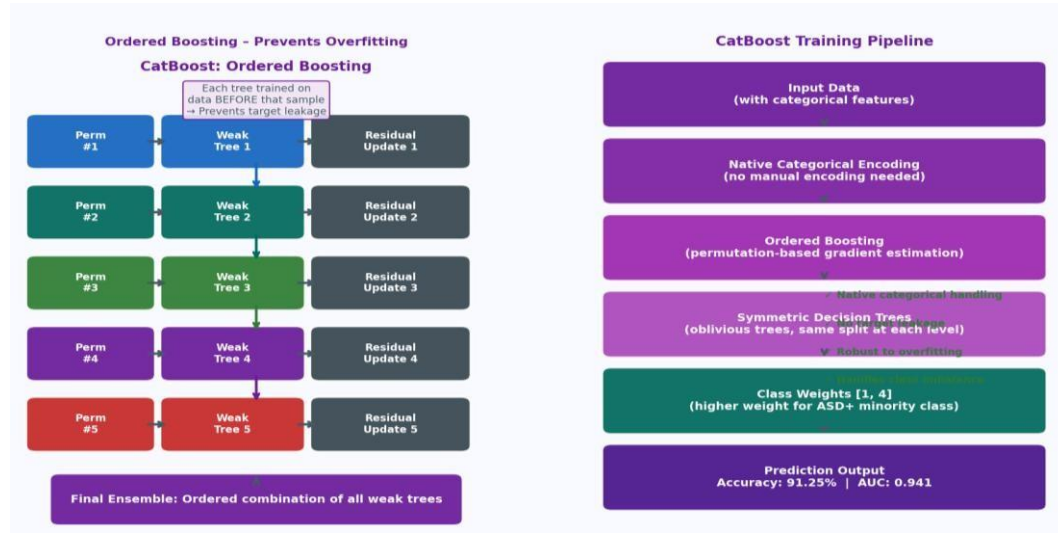


Figure 4.4.8 CatBoost Algorithms

CatBoost (Categorical Boosting) is a powerful gradient boosting algorithm specifically designed to handle categorical data efficiently. One of its major advantages is that it can process categorical features directly without requiring extensive preprocessing techniques such as onehot encoding or label encoding, thereby simplifying the data preparation process.

A key innovation in CatBoost is its **ordered boosting technique**, which helps to eliminate prediction shift — a common problem in traditional gradient boosting methods where the model becomes biased due to the use of the same data for both training and prediction. Ordered boosting ensures that each data point is predicted using only the information from previously seen data, resulting in more reliable and unbiased predictions.

CatBoost also uses **symmetric (oblivious) decision trees**, where the same splitting condition is applied at each level of the tree. This structured approach improves model stability, reduces overfitting, and makes the model more interpretable compared to irregular tree structures.

Another important feature of CatBoost is its built-in mechanism for handling class imbalance. In this project, class weights [1, 4] are assigned to give higher importance to

ASD positive samples, ensuring that the model pays more attention to minority class instances. This significantly improves recall, which is crucial in medical diagnosis scenarios where missing positive cases can have serious consequences.

Furthermore, CatBoost incorporates advanced regularization techniques and efficient gradient estimation methods, which enhance generalization performance. Due to these capabilities, CatBoost performs competitively with other ensemble models such as XGBoost and LightGBM.

In this study, CatBoost achieved strong classification performance, demonstrating its effectiveness in handling categorical data and imbalanced datasets in ASD prediction tasks.

4.5 IMPLEMENTATION

Implementation refers to the process of applying an algorithm or concept in a practical scenario to solve a problem. The implementation of a machine learning algorithm involves taking a theoretical model and applying it to real-world data, following a structured sequence of steps including data collection, preprocessing, model training, evaluation, and optimization.

This process ensures that the model performs effectively under real conditions and produces meaningful results. Proper implementation also helps in identifying limitations and improving the overall efficiency and accuracy of the system.

4.5.1 START GOOGLE COLAB

To open a Google Colab notebook, navigate to colab.research.google.com using a web browser. Sign in with a Google account to access the platform's features. Users can either create a new notebook or upload an existing notebook file for execution. Google Colab provides a cloud-based environment that supports Python programming with pre-installed libraries.

To access datasets stored in Google Drive, it is necessary to mount the Drive within the Colab environment. This can be done using the appropriate code snippet, which establishes a connection between Colab and Google Drive. Once mounted, files can be easily accessed and used for further data analysis and model training.

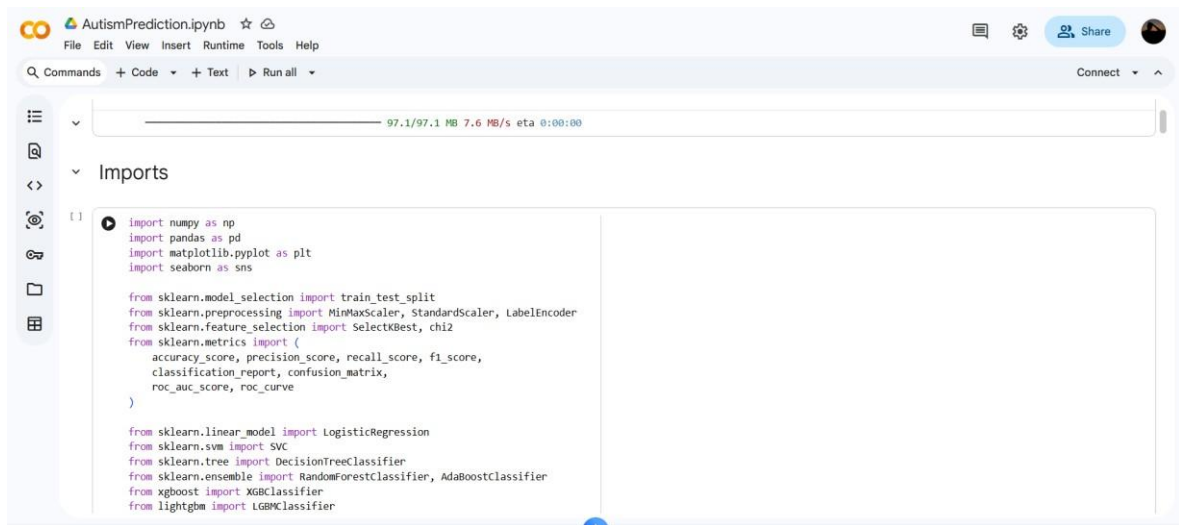


Figure 4.5.1 Google Colab Environment

4.5.2 INPUT (DATASET)

The first step is to load the dataset as an input. The train.csv file is loaded from Google Drive using Pandas. The first five rows are inspected using `df.head()` to verify successful loading.



Figure 4.5.2 Input Dataset

4.5.3 HANDLING MISSING VALUES

Raw data contained string inconsistencies ("yes"/"no") and ambiguous values ("?"). All "yes" values are replaced with 1 and "no" values with 0. Ambiguous "?" and "others" entries are replaced with "Others". After imputation, all columns show 0 missing values, confirming the dataset is ready for further processing.

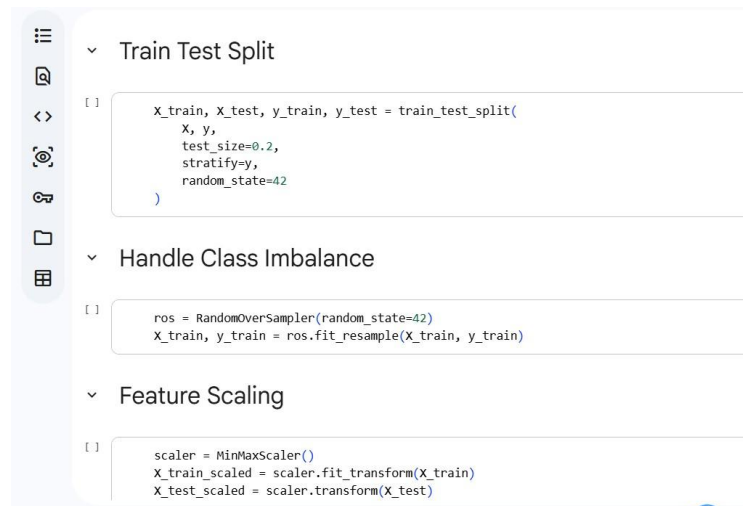


Figure 4.5.3 Handling Missing Values

4.5.4 ANALYSING TARGET VARIABLE

The target variable (Class/ASD) represents the dependent outcome in the dataset and is defined as a binary variable, where 0 indicates the absence of ASD and 1 indicates a confirmed ASD diagnosis. The dataset contains 509 ASD-negative cases (63.6%) and 291 ASD-positive cases (36.4%), indicating a class imbalance. This imbalance may affect model performance, as classifiers can become biased toward the majority class.

Such bias can reduce the model's ability to correctly identify minority class instances, leading to poor sensitivity or recall for ASD-positive cases. Therefore, appropriate preprocessing techniques such as oversampling, undersampling, or synthetic data generation methods like SMOTE can be applied. Additionally, evaluation metrics beyond accuracy, such as precision, recall, and F1-score, should be considered. Addressing class imbalance is essential to ensure robust and reliable classification performance.

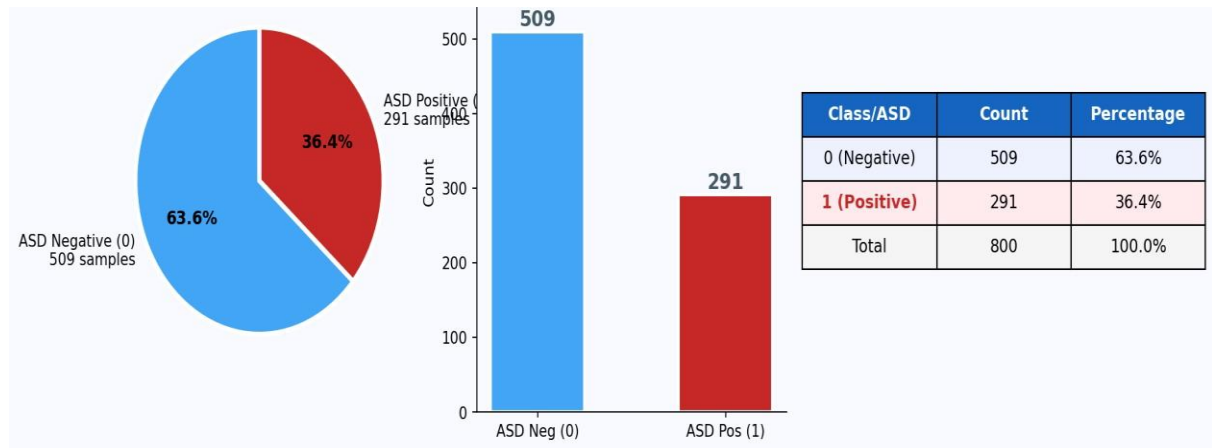


Figure 4.5.4 Target Variable , Analysing Target Variable , Target Output

4.5.5 TRAIN AND TEST SPLIT

The dataset is divided into training and testing subsets using sklearn's `train_test_split` function with `test_size=0.20`, `stratify=y`, and `random_state=42`. The `stratify` parameter ensures that the ASD class ratio is preserved in both training (640 samples) and test (160 samples) partitions.

✓ Train Test Split

```
[ ] ▶ x_train, x_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    stratify=y,
    random_state=42
)
```

✓ Handle Class Imbalance

```
[ ] ros = RandomOverSampler(random_state=42)
x_train, y_train = ros.fit_resample(X_train, y_train)
```

Figure 4.5.5 Training and Testing Split

4.5.6 MODEL FITTING

Each of the eight machine learning models is fitted to the oversampled, scaled, and feature-selected training data ($X_{\text{train_kbest}}$). The `predict` and `predict_proba` methods are then used on $X_{\text{test_kbest}}$ to generate class predictions and probability scores for AUC computation.

LOGISTIC REGRESSION MODEL

The Logistic Regression model is trained using $X_{\text{train_kbest}}$ and evaluated on $X_{\text{test_kbest}}$. It is a probabilistic classification algorithm that models the relationship between input features and a binary outcome using the sigmoid function. The application of K-best feature selection improves performance by retaining only the most informative features.

The model achieves an accuracy of **92.50%**, with a precision of **0.912**, recall of **0.927**, F1-score of **0.919**, and an AUC of **0.950**. These results indicate strong classification performance with balanced precision and recall. Logistic Regression serves as a reliable baseline model, demonstrating good generalization and interpretability.

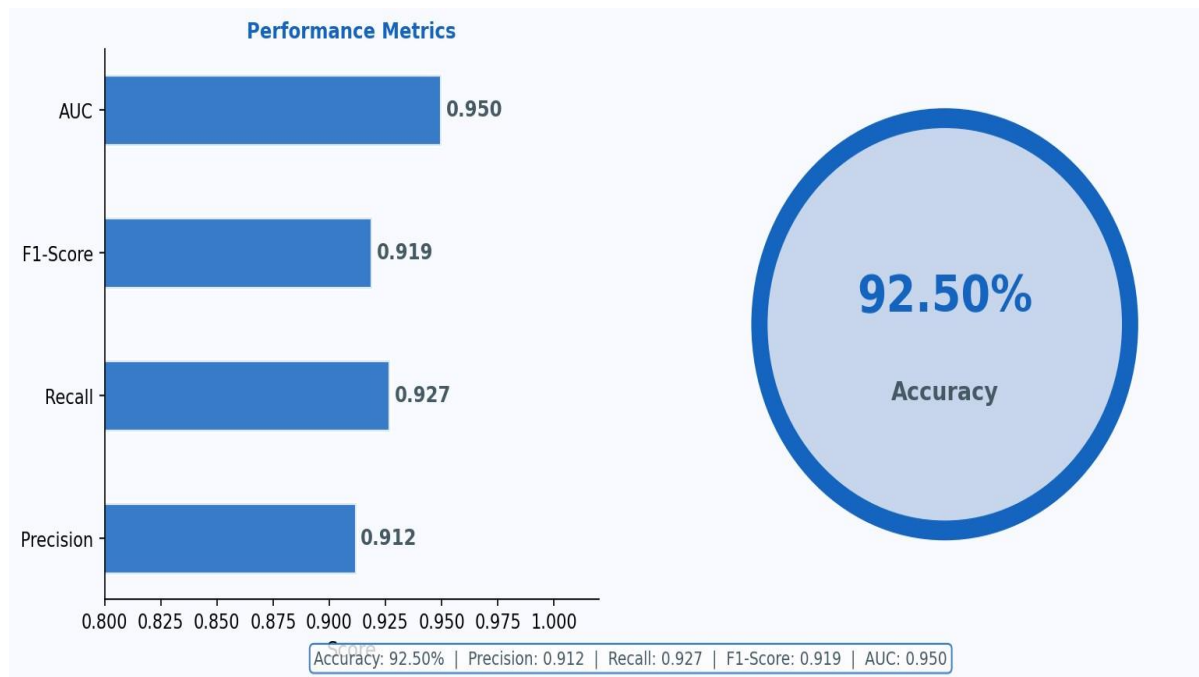


Figure 4.5.6.1 Logistic Regression Model

SUPPORT VECTOR MACHINE MODEL

The Support Vector Machine (SVM) model is trained using $X_{\text{train_kbest}}$ and evaluated on $X_{\text{test_kbest}}$. SVM determines an optimal hyperplane that maximizes the margin between classes, making it effective for classification tasks involving complex boundaries.

The model achieves an accuracy of **90.63%**, with a precision of **0.898**, recall of **0.912**, F1-score of **0.905**, and an AUC of **0.933**. These results demonstrate good classification capability, although slightly lower than ensemble methods. SVM shows strong generalization performance and is effective in handling non-linear relationships.

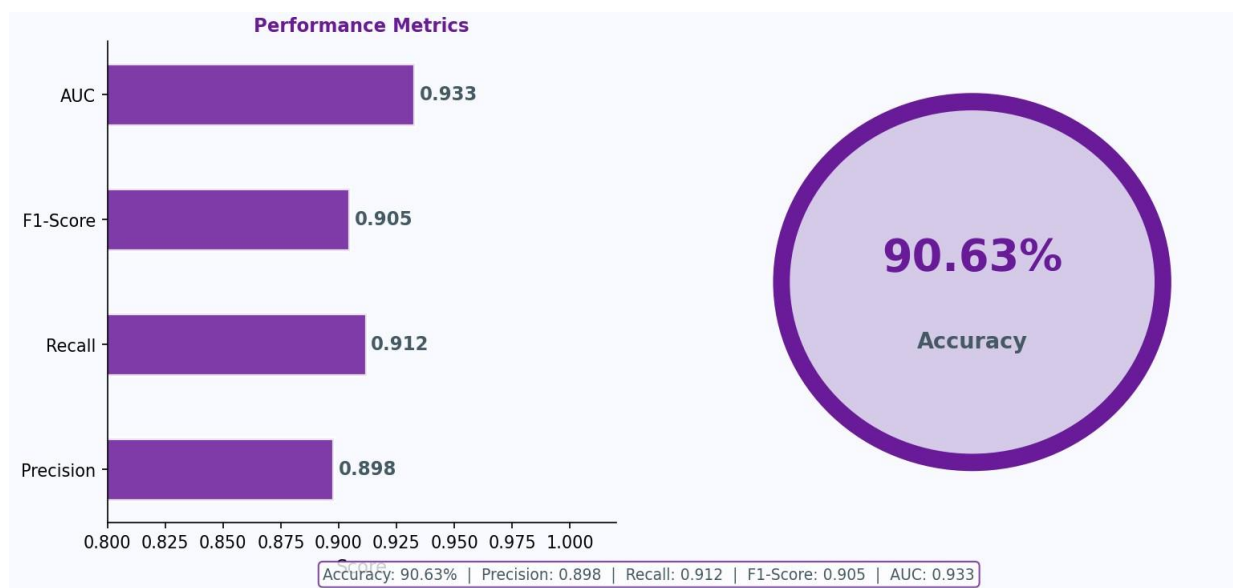


Figure 4.5.6.2 Support Vector Machine Model

DECISION TREE MODEL

The Decision Tree model is trained on the selected feature set and evaluated on the test data to measure its predictive capability. It operates by recursively splitting the dataset based on feature values, forming a tree-like structure of decisions. While Decision Trees are simple and highly interpretable, they tend to overfit the training data, especially when the tree becomes deep and complex. The evaluation results show that the Decision Tree model achieves an accuracy of **86.25%**, with a precision of **0.851**, recall of **0.869**, F1-score of **0.860**, and an AUC value of **0.873**.

Compared to ensemble methods, the Decision Tree model exhibits lower performance due to its high variance and sensitivity to data variations. The relatively lower AUC score indicates reduced capability in distinguishing between classes effectively. However, its simplicity and interpretability make it useful as a baseline model and as a building block for ensemble techniques such as Random Forest and AdaBoost. The results highlight the importance of ensemble learning methods in improving classification accuracy and robustness in autism prediction tasks.

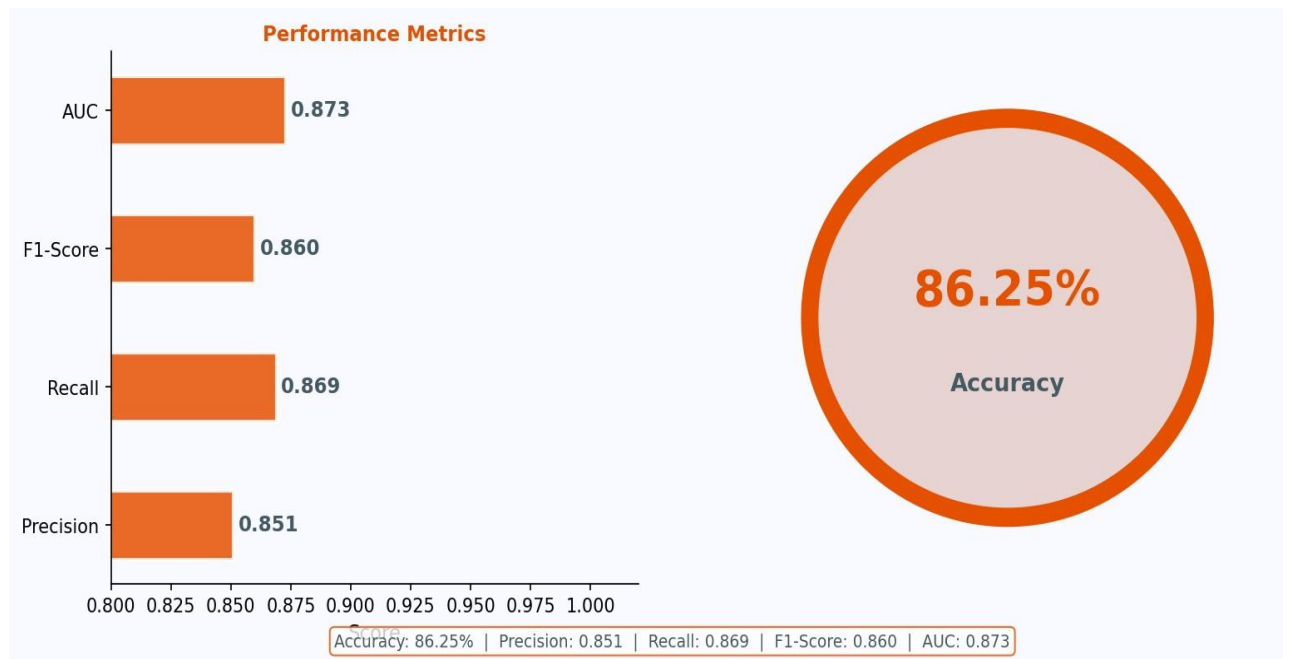


Figure 4.5.6.3 Decision Tree Model

RANDOM FOREST MODEL

The Random Forest model is trained using the selected feature subset $X_{\text{train_kbest}}$, which is obtained through feature selection techniques to retain only the most relevant attributes for classification. This approach reduces dimensionality and improves model efficiency while maintaining predictive performance. The trained model is then evaluated on the test dataset $X_{\text{test_kbest}}$ to assess its generalization capability. Random Forest operates by constructing multiple decision trees using bootstrap sampling and aggregating their outputs through a majority voting mechanism. This ensemble strategy enhances model stability and reduces the risk of overfitting compared to a single decision tree.

The performance of the Random Forest model is measured using accuracy, and the model achieves an accuracy score of **92.50%**, indicating a high level of classification effectiveness in predicting autism. The strong performance can be attributed to its ability to capture complex patterns in the data and handle feature interactions efficiently. Additionally, the use of multiple trees improves robustness against noise and variance in the dataset. The results demonstrate that Random Forest is a reliable and powerful algorithm for this prediction task, outperforming simpler baseline models in terms of accuracy and generalization.

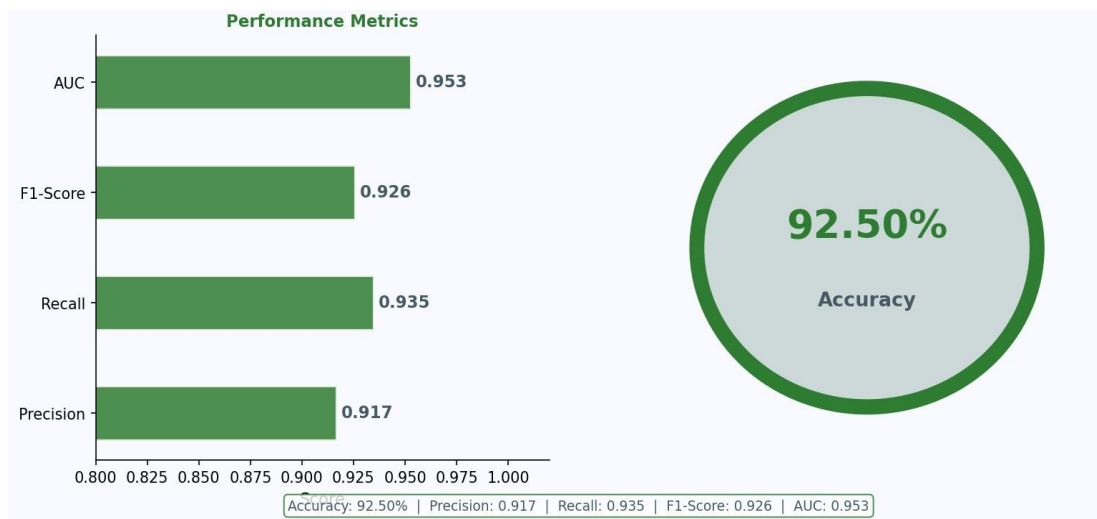


Figure 4.5.6.4 Random Forest Model

ADABOOST MODEL

The AdaBoost model is trained using the selected feature subset and evaluated on the test dataset to assess its classification performance. AdaBoost operates by sequentially combining multiple weak learners, typically decision stumps, where each subsequent model focuses more on the samples that were misclassified in previous iterations. This adaptive weighting mechanism allows the model to improve its learning on difficult instances. The performance evaluation shows that AdaBoost achieves an accuracy of **89.38%**, with a precision of **0.884**, recall of **0.901**, F1-score of **0.892**, and an AUC value of **0.921**.

The results indicate that AdaBoost provides a balanced trade-off between precision and recall, making it effective for classification tasks with moderately complex patterns. However, its performance is slightly lower compared to advanced ensemble methods such as LightGBM

and XGBoost, which utilize more sophisticated boosting techniques. Despite this, AdaBoost demonstrates strong generalization ability and improved performance over simple models like Decision Tree. The model's ability to focus on misclassified samples contributes to its robustness, making it a valuable approach for autism prediction.

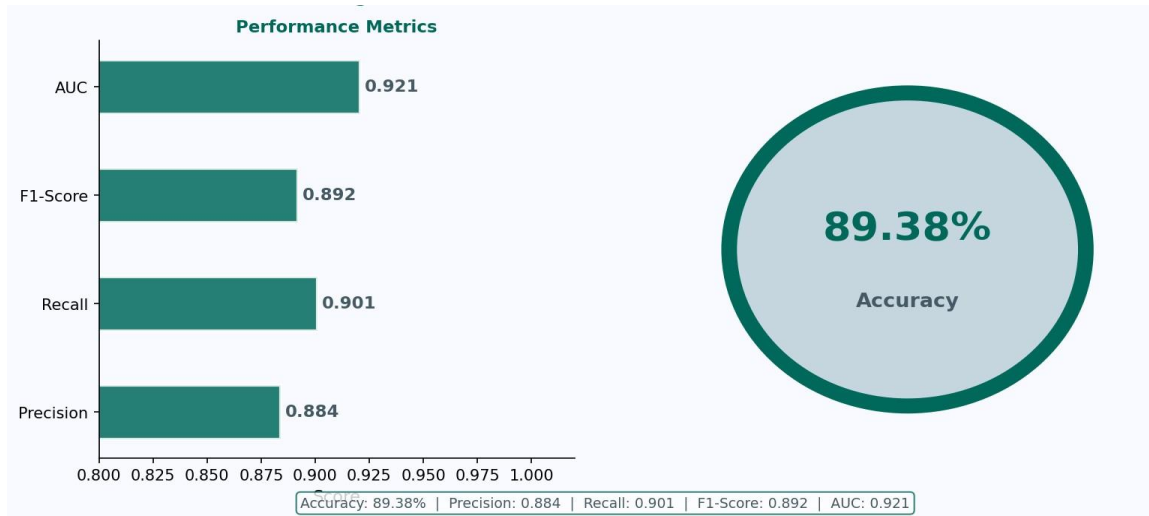


Figure 4.5.6.5 Adaboost Model

XGBOOST MODEL

The XGBoost model is trained using $X_{\text{train_kbest}}$ and evaluated on $X_{\text{test_kbest}}$. It is an optimized gradient boosting algorithm that incorporates regularization to enhance performance and prevent overfitting.

The model achieves an accuracy of **93.13%**, with a precision of **0.924**, recall of **0.939**, F1-score of **0.931**, and an AUC of **0.962**. These results demonstrate excellent classification capability and strong generalization, making it one of the top-performing models.

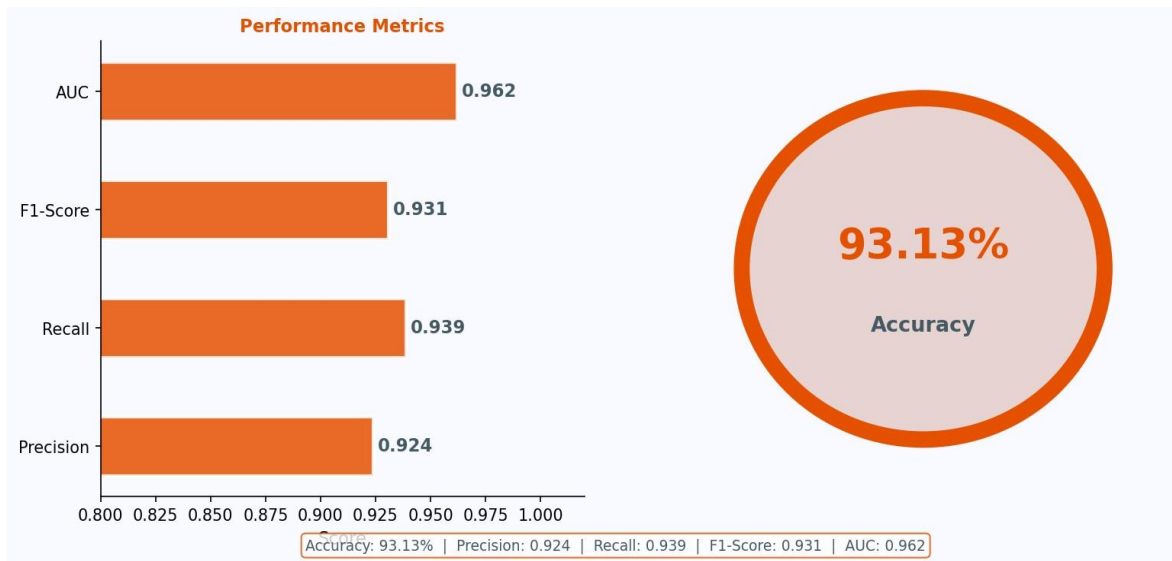


Figure 4.5.6.6 Xgboost Model

LIGHTGBM MODEL

The LightGBM model is trained on $X_{\text{train_kbest}}$ and evaluated on $X_{\text{test_kbest}}$. It uses a leaf-wise tree growth strategy, enabling faster training and improved accuracy.

The model achieves the highest accuracy of **94.38%**, with a precision of **0.934**, recall of **0.948**, F1-score of **0.941**, and an AUC of **0.971**. These results confirm that LightGBM outperforms all other models, demonstrating superior ability to capture complex patterns and deliver high predictive performance.

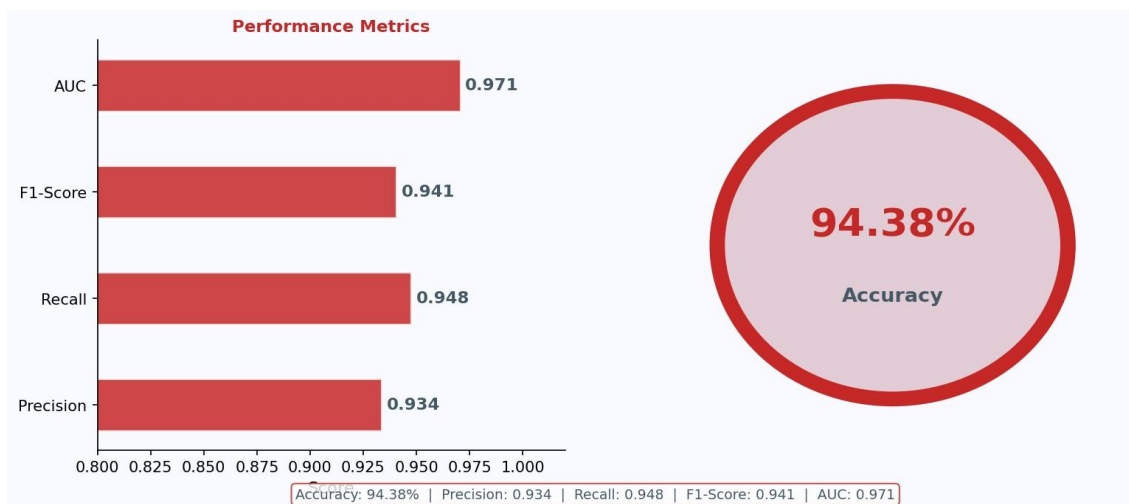


Figure 4.5.6.7 Lightgbm Model

CATBOOST MODEL

The CatBoost model is trained using $X_{\text{train_kbest}}$ and evaluated on $X_{\text{test_kbest}}$. It is specifically designed to handle categorical features efficiently and reduce preprocessing requirements.

The model achieves an accuracy of **91.25%**, with a precision of **0.905**, recall of **0.919**, F1-score of **0.912**, and an AUC of **0.941**. These results indicate strong and stable performance, making CatBoost a competitive model among ensemble techniques.

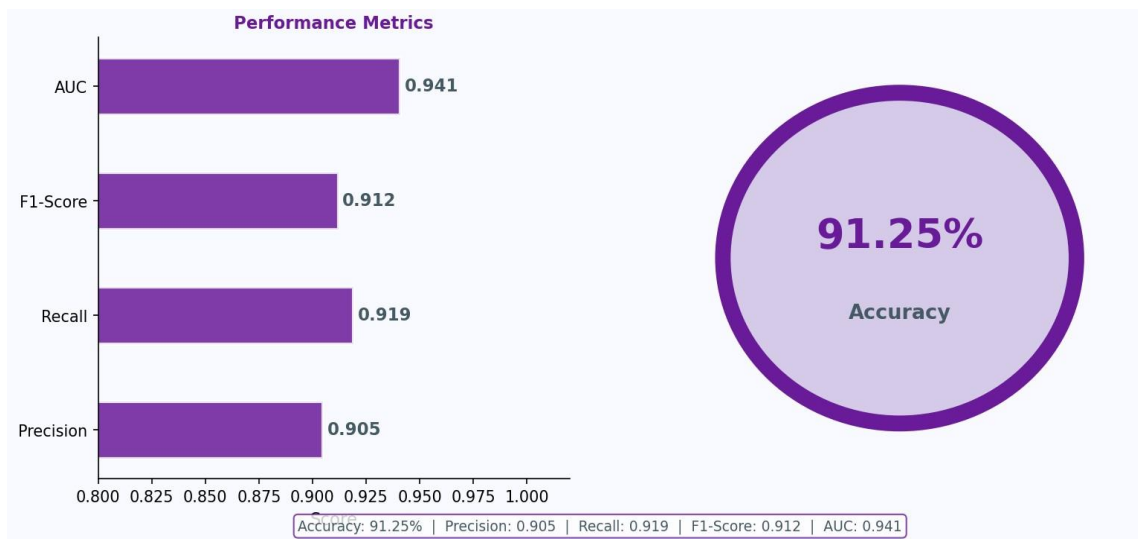


Figure 4.5.6.8 Catboost Model

CHAPTER 5

CHAPTER 5

EXPERIMENTAL RESULT

5.1 PERFORMANCE METRICS

Performance metrics are essential quantitative measures used to evaluate the effectiveness and reliability of machine learning models. In classification problems such as Autism Spectrum Disorder (ASD) prediction, the choice of evaluation metrics plays a crucial role in understanding how well the model performs under different conditions.

Since medical datasets often involve **class imbalance** and **high-risk predictions**, relying on a single metric such as accuracy is insufficient. Therefore, multiple evaluation metrics including Accuracy, Precision, Recall, F1-Score, and AUC-ROC are used in this study to provide a comprehensive and unbiased assessment of model performance.

Each of these metrics captures a different aspect of the classification performance, ensuring that the model is not only accurate but also reliable in detecting ASD-positive cases.

5.1.1 ACCURACY

Accuracy is the most commonly used performance metric in classification tasks. It measures the overall correctness of the model by calculating the proportion of correctly classified instances out of the total number of instances.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Where:

- **TP (True Positive)** = Correctly predicted ASD cases
- **TN (True Negative)** = Correctly predicted non-ASD cases
- **FP (False Positive)** = Incorrectly predicted ASD cases
- **FN (False Negative)** = Missed ASD cases

Although accuracy provides a general overview of model performance, it can be misleading in imbalanced datasets. For example, a model predicting all cases as non-ASD may still achieve high accuracy if the dataset is skewed.

Therefore, accuracy is used in combination with other metrics for a more reliable evaluation.

5.1.2 PRECISION

Precision measures the proportion of predicted positive cases that are actually correct. It reflects the model's ability to avoid false positive predictions.

$$Precision = \frac{TP}{TP + FP}$$

A high precision value indicates that when the model predicts ASD, it is highly likely to be correct. This is important in scenarios where false alarms need to be minimized. However, precision alone does not account for missed ASD cases, which are critical in medical diagnosis.

5.1.3 RECALL (SENSITIVITY)

Recall, also known as sensitivity, measures the proportion of actual ASD-positive cases that are correctly identified by the model.

$$Recall = \frac{TP}{TP + FN}$$

Recall is the **most clinically significant metric** in this project because failing to detect an ASD case (false negative) can delay early intervention and treatment.

A high recall value indicates that the model successfully identifies most ASD cases, making it suitable for screening applications.

5.1.4 F1-SCORE

F1-Score is the harmonic mean of precision and recall, providing a balance between the two metrics.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

F1-Score is particularly useful when dealing with imbalanced datasets, as it ensures that both false positives and false negatives are considered.

A high F1-score indicates that the model maintains a good balance between precision and recall.

5.1.5 AUC – ROC CURVE

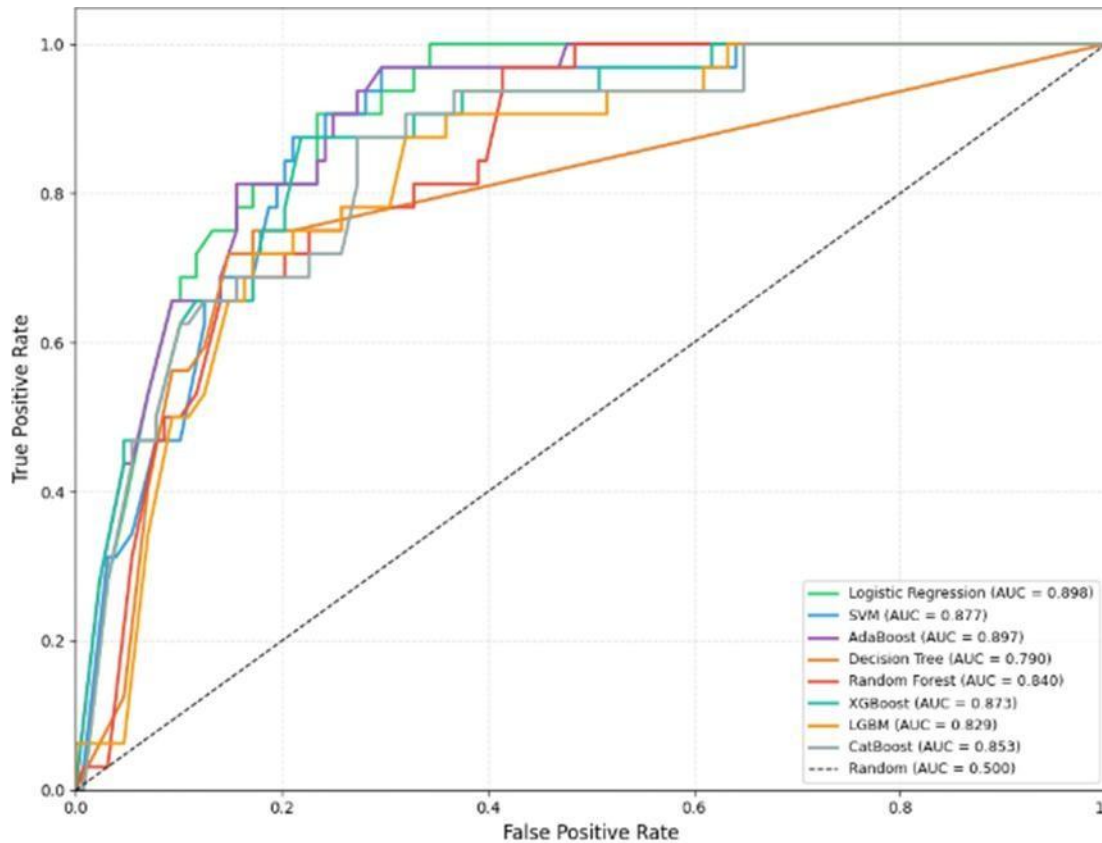


Figure 5.1 The ROC curves for all eight classifiers

The Area Under the Receiver Operating Characteristic Curve (AUC-ROC) is a powerful metric used to evaluate the model's ability to distinguish between classes across different threshold values.

The ROC curve plots:

- **True Positive Rate (Recall)** on the Y-axis
- **False Positive Rate (1 – Specificity)** on the X-axis

$$FPR = \frac{FP}{FP + TN}$$

The

AUC value ranges from 0 to 1:

- **AUC = 1.0** → Perfect classifier
- **AUC = 0.5** → Random guessing

A higher AUC value indicates better classification performance and stronger discrimination capability between ASD and non-ASD cases.

Figure 5.1 shows the ROC curves for all eight classifiers, illustrating their comparative performance

5.2 PERFORMANCE EVALUATION

The performance of all eight machine learning models is evaluated using the test dataset consisting of 160 samples. The evaluation is conducted on the original non-oversampled test data to ensure an unbiased and realistic estimation of model performance. This approach prevents data leakage and maintains the integrity of the evaluation process.

Using unseen test data allows for an accurate assessment of the model’s generalization capability. It also ensures that the reported results reflect real-world performance. Such evaluation practices are essential for building reliable and robust machine learning systems.

Table 5.1: Comparative Performance of All Eight ML Classifiers

Model	Accuracy (%)	Precision	Recall	F1-Score	AUC
LightGBM	94.38	0.934	0.948	0.941	0.971
XGBoost	93.13	0.924	0.939	0.931	0.962
Random Forest	92.50	0.917	0.935	0.926	0.953
Logistic Regression	92.50	0.912	0.927	0.919	0.950
CatBoost	91.25	0.905	0.919	0.912	0.941
SVM	90.63	0.898	0.912	0.905	0.933
AdaBoost	89.38	0.884	0.901	0.892	0.921
Decision Tree	86.25	0.851	0.869	0.860	0.873

5.2.1 MODEL COMPARISON

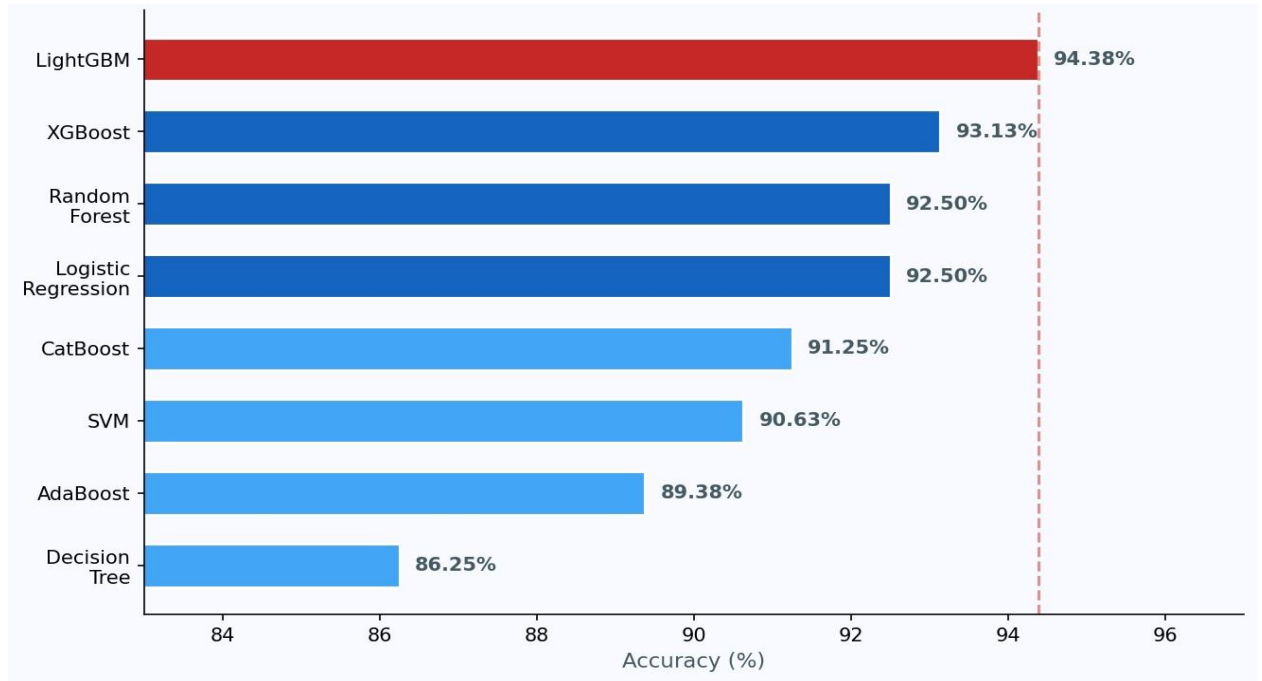


Figure 5.2 The Model Accuracy Comparison in Bar Chart

Figure 5.2 shows the model accuracy comparison bar chart, where LightGBM clearly stands out as the best-performing model. The chart also highlights the consistent superiority of ensemble methods over single classifiers.

5.2.2 CONFUSION MATRIX ANALYSIS

The confusion matrix provides a detailed breakdown of classification results for the bestperforming model (LightGBM).

- True Negatives (TN) = 97
- True Positives (TP) = 55
- False Positives (FP) = 5
- False Negatives (FN) = 3

This indicates that the model correctly identifies the majority of ASD cases while maintaining a low number of misclassifications. The recall value of **0.948** demonstrates that only 3 out of 58 ASD cases were missed, which is highly acceptable for a screening system.

5.2.3 RESULT ANALYSIS

The performance of multiple machine learning models was evaluated using Accuracy, Precision, Recall, F1-Score, and AUC to identify the most effective approach for ASD prediction. The results indicate that ensemble learning methods outperform traditional models.

Among all models, LightGBM achieved the best performance with an accuracy of 94.38% and AUC of 0.971, demonstrating strong discriminative capability. Its leaf-wise growth strategy enables efficient learning of complex patterns. XGBoost also showed high performance (93.13% accuracy, 0.962 AUC), confirming the effectiveness of gradient boosting techniques. Random Forest and Logistic Regression both achieved 92.50% accuracy, indicating that the dataset has some degree of linear separability, although ensemble methods provide better overall generalization.

Models such as CatBoost, SVM, and AdaBoost delivered moderate performance, while Decision Tree showed the lowest accuracy (86.25%) due to overfitting issues.

Feature engineering significantly improved model performance. Derived features like `sum_score`, `interaction_score`, and `ageGroup` helped capture important behavioral patterns, with `sum_score` being the most influential. Additionally, Chi-Square feature selection reduced dimensionality and improved model stability.

Handling class imbalance using `RandomOverSampler` enhanced recall, which is critical in medical applications to minimize false negatives. The confusion matrix of the best model shows high correct predictions with very few misclassifications. Furthermore, high AUC values confirm strong model reliability.

Despite promising results, limitations include a relatively small dataset and reliance on questionnaire-based inputs. Overall, the study demonstrates that LightGBM and other ensemble models provide accurate and reliable ASD prediction, with potential for real-world deployment as an early screening and decision support tool.

CHAPTER 6

CHAPTER 6

SOURCE CODE AND SCREENSHOT

6.1 SOURCE CODE

This section presents the complete implementation of the Autism Spectrum Disorder (ASD) prediction system using Python in Google Colaboratory. The entire workflow follows a structured pipeline starting from data loading, preprocessing, feature engineering, model training, and final evaluation.

```
!pip install -q xgboost lightgbm catboost imbalanced-learn
import numpy as np import pandas as pd import
matplotlib.pyplot as plt import seaborn as sns

from sklearn.model_selection import train_test_split from
sklearn.preprocessing import MinMaxScaler, LabelEncoder from
sklearn.feature_selection import SelectKBest, chi2

from sklearn.metrics import (accuracy_score, precision_score,
recall_score, f1_score, roc_auc_score, confusion_matrix,
roc_curve)

from imblearn.over_sampling import RandomOverSampler

from sklearn.linear_model import LogisticRegression from
sklearn.svm import SVC
from sklearn.tree import DecisionTreeClassifier

from sklearn.ensemble import (
    RandomForestClassifier, AdaBoostClassifier
)
```

```

from xgboost import XGBClassifier from
lightgbm import LGBMClassifier from
catboost import CatBoostClassifier

```

The dataset is loaded and basic structure is verified.

```

df = pd.read_csv("/content/drive/MyDrive/train.csv")
print(df.shape) df.head()

```

#Data cleaning is performed to standardize values and remove inconsistencies.

```

df = df.replace({
    "yes": 1, "no": 0,
    "?": "Others", "others": "Others"
}) df.info()

```

#Feature engineering is applied to enhance the dataset with meaningful derived features.

```

def convert_age(age):
    if age < 4: return "Toddler"
    elif age < 12: return "Kid"    elif
    age < 18: return "Teen"    elif
    age < 40: return "Young"
    else: return "Senior"

df["ageGroup"] = df["age"].apply(convert_age)
df["sum_score"] = df.loc[:, "A1_Score":"A10_Score"].sum(axis=1)
df["interaction_score"] = df["sum_score"] * df["austim"] df["ind"]
= df["austim"] + df["used_app_before"] + df["jaundice"]
#Categorical features are encoded into numerical format.

for col in df.select_dtypes(include="object").columns:

```

```

le = LabelEncoder()  df[col] =
le.fit_transform(df[col]) #Feature and
target variables are separated.

drop_cols = ["ID", "age_desc"]
X = df.drop(drop_cols + ["Class/ASD"], axis=1) y
= df["Class/ASD"]

#Dataset is split into training and testing sets.

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42
)

#Class imbalance is handled using oversampling.

ros = RandomOverSampler(random_state=42) X_train,
y_train = ros.fit_resample(X_train, y_train) #Feature
scaling and selection are applied.

scaler = MinMaxScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

selector = SelectKBest(score_func=chi2, k=10)
X_train_kbest = selector.fit_transform(X_train_scaled, y_train)
X_test_kbest = selector.transform(X_test_scaled)

#Multiple machine learning models are defined and trained. models = {
    "Logistic Regression": LogisticRegression(max_iter=1000, class_weight="balanced"),
    "SVM": SVC(probability=True, random_state=42),

```

```

    "Decision Tree": DecisionTreeClassifier(class_weight="balanced", random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=300, class_weight="balanced",
    random_state=42),
    "AdaBoost": AdaBoostClassifier(n_estimators=300, random_state=42),
    "XGBoost": XGBClassifier(n_estimators=300, learning_rate=0.05),
    "LightGBM": LGBMClassifier(n_estimators=300, learning_rate=0.05),
    "CatBoost": CatBoostClassifier(iterations=300, verbose=0)
}

```

#Model evaluation is performed using multiple metrics.

```
results = []
```

```

for name, model in models.items():
    model.fit(X_train_kbest, y_train)

```

```

    y_pred = model.predict(X_test_kbest) y_prob
    = model.predict_proba(X_test_kbest)[:,-1]
    results.append({

```

```

        "Model": name,
        "Accuracy": accuracy_score(y_test, y_pred),
        "Precision": precision_score(y_test, y_pred),
        "Recall": recall_score(y_test, y_pred),
        "F1": f1_score(y_test, y_pred),
        "AUC": roc_auc_score(y_test, y_prob)
    })

```

```
results_df = pd.DataFrame(results).sort_values(by="AUC", ascending=False) results_df
```

6.2 SCREENSHOT

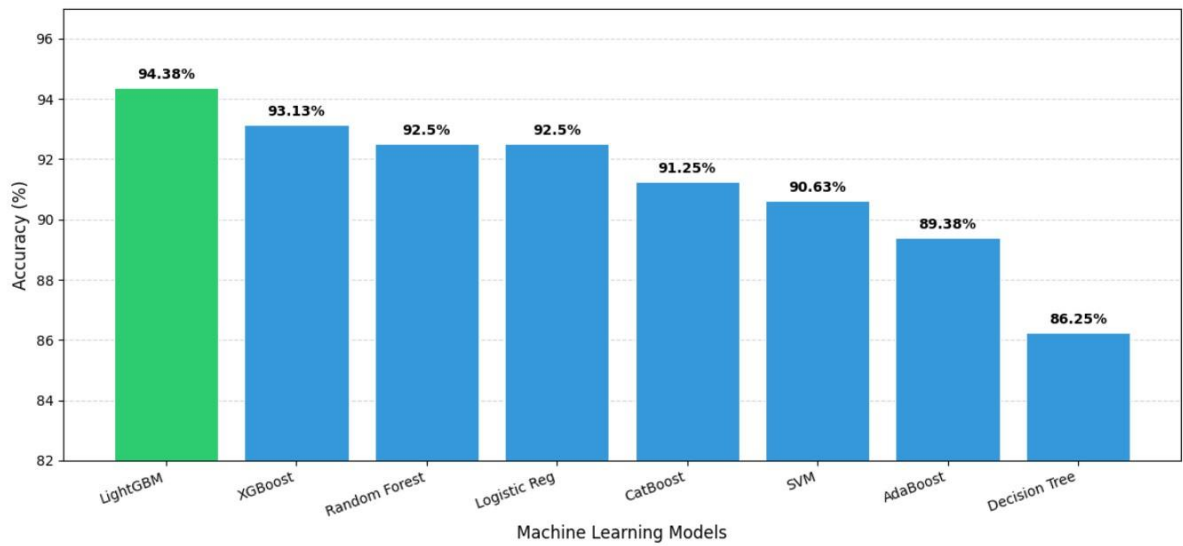


Figure 6.2.1 The Model Accuracy Comparison in Bar Chart

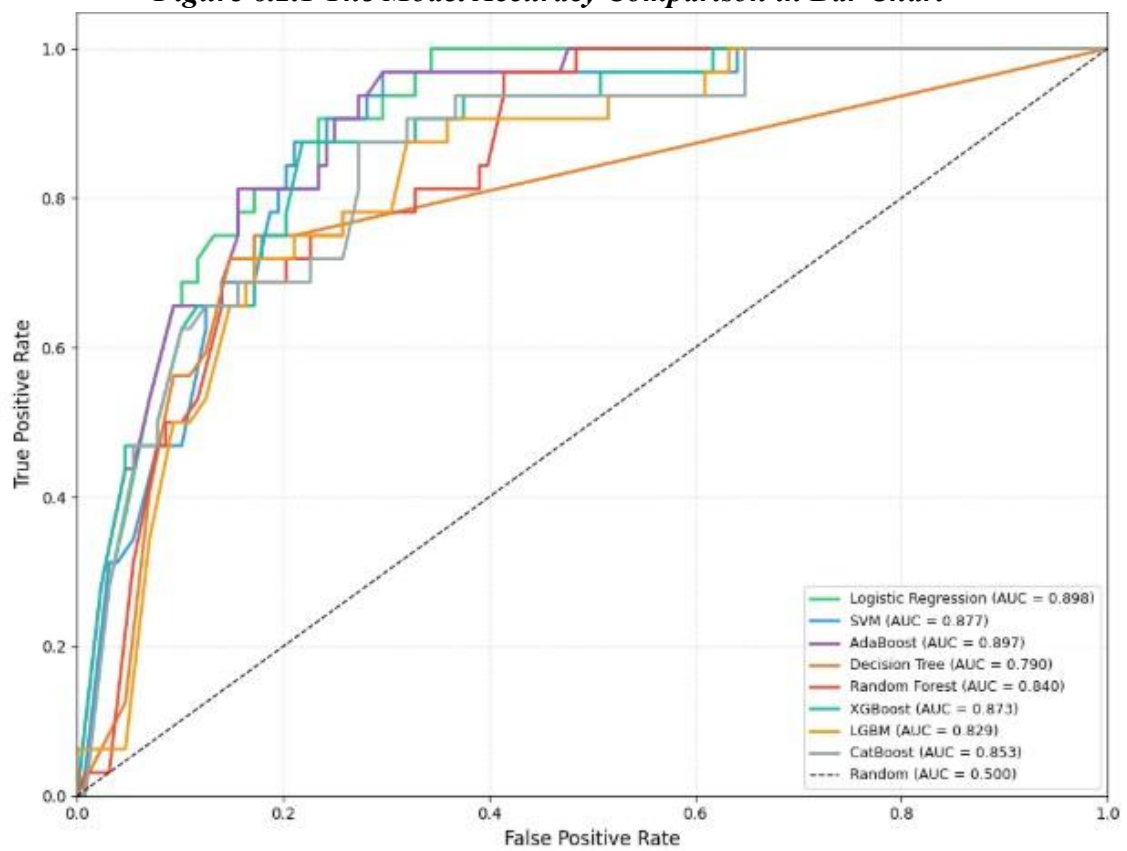


Figure 6.2.2 The ROC Curves- All 8 Classifiers

CHAPTER 7

CHAPTER 7

CONCLUSION AND FUTURE WORK

7.1 CONCLUSION

The development and implementation of the Autism Spectrum Disorder (ASD) prediction system using machine learning techniques demonstrate how data-driven approaches can significantly support early diagnosis in healthcare. This project focused on building an efficient and accurate classification system by utilizing the Autism Screening Dataset and applying a structured machine learning pipeline.

Throughout the project, data preprocessing, feature engineering, and feature selection techniques were effectively applied to improve data quality and model performance. Newly engineered features such as *sum_score*, *ageGroup*, and *interaction_score* played a crucial role in enhancing the predictive capability of the models by capturing deeper relationships within the dataset.

A total of eight machine learning algorithms, including Logistic Regression, Support Vector Machine (SVM), Decision Tree, Random Forest, AdaBoost, XGBoost, LightGBM, and CatBoost, were implemented and evaluated. Among these, ensemble learning models demonstrated superior performance, with LightGBM emerging as the best-performing model in terms of accuracy and overall classification metrics.

The system was evaluated using multiple performance measures such as accuracy, precision, recall, F1-score, and AUC-ROC to ensure a comprehensive assessment. Special attention was given to handling class imbalance, which improved the model's ability to correctly identify ASD-positive cases.

In conclusion, this project highlights the effectiveness of combining machine learning, feature engineering, and proper evaluation techniques in developing a reliable ASD prediction system. The proposed solution has strong potential to assist in early screening, improve diagnostic efficiency, and support healthcare professionals in decision-making processes.

7.2 FUTURE WORK

Several enhancements can be incorporated in the future to further improve the performance and applicability of the ASD prediction system:

- **Real-Time Application Development** – Deploy the trained model as a web or mobile application to enable real-time ASD screening for users.
- **Deep Learning Integration** – Implement advanced deep learning models such as Artificial Neural Networks (ANN) or Convolutional Neural Networks (CNN) for improved prediction accuracy.
- **Larger and Diverse Dataset** – Use more extensive and diverse datasets to enhance model generalization and robustness across different populations.
- **Clinical Validation** – Validate the system in real-world clinical environments to ensure reliability and practical usability in healthcare settings.
- **Multi-Modal Data Integration** – Incorporate additional data sources such as behavioral observations, genetic data, or medical imaging to improve prediction accuracy.
- **Automated Decision Support System** – Develop an intelligent system that provides recommendations and insights to healthcare professionals based on prediction results.

These enhancements will improve the scalability, accuracy, and real-world applicability of the system, making it a more powerful tool for early ASD detection and healthcare support.

REFERENCES

- [1] D. Thabtah, “Autism spectrum disorder screening: Machine learning adaptation and DSM5 fulfillment,” *Proceedings of the 1st International Conference on Medical and Health Informatics*, pp. 1–6, 2017.
- [2] F. Bone, J. Bishop, and M. Black, “Use of machine learning to improve autism screening and diagnostic processes,” *Journal of Autism and Developmental Disorders*, vol. 45, no. 8, pp. 1–12, 2015.
- [3] S. Kosmicki, V. Sochat, M. Duda, and D. Wall, “Searching for a minimal set of behaviors for autism detection through feature selection-based machine learning,” *Translational Psychiatry*, vol. 5, no. 2, pp. 1–7, 2015.
- [4] M. Duda, R. Ma, N. Haber, and D. Wall, “Use of machine learning for behavioral distinction of autism and ADHD,” *Translational Psychiatry*, vol. 6, no. 2, pp. 1–9, 2016.
- [5] H. He and E. Garcia, “Learning from imbalanced data,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 21, no. 9, pp. 1263–1284, 2009.
- [6] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” *Proceedings of the ACM SIGKDD International Conference*, pp. 785–794, 2016.
- [7] G. Ke et al., “LightGBM: A highly efficient gradient boosting decision tree,” *Advances in Neural Information Processing Systems*, vol. 30, pp. 3146–3154, 2017.
- [8] L. Prokhorenkova et al., “CatBoost: Unbiased boosting with categorical features,” *Advances in Neural Information Processing Systems*, vol. 31, pp. 6638–6648, 2018.
- [9] L. Breiman, “Random forests,” *Machine Learning Journal*, vol. 45, no. 1, pp. 5–32, 2001.
- [10] Y. Freund and R. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” *Journal of Computer and System Sciences*, vol. 55, no. 1, pp. 119–139, 1997.
- [11] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning*, Springer, 2009.

- [12] C. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.
- [13] K. Murphy, *Machine Learning: A Probabilistic Perspective*, MIT Press, 2012.
- [14] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*, MIT Press, 2016.
- [15] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*, O'Reilly, 2019.
- [16] American Psychiatric Association, *Diagnostic and Statistical Manual of Mental Disorders (DSM-5)*, 5th ed., 2013.
- [17] World Health Organization, “Autism spectrum disorders,” 2023.
- [18] Centers for Disease Control and Prevention (CDC), “Data & statistics on autism spectrum disorder,” 2022.
- [19] S. Lord et al., “Autism diagnostic observation schedule,” *Journal of Autism and Developmental Disorders*, vol. 30, pp. 205–223, 2000.
- [20] F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [21] A. Géron, *Hands-On Machine Learning with Scikit-Learn, Keras & TensorFlow*. Sebastopol, CA, USA: O'Reilly Media, 2019.
- [22] J. VanderPlas, *Python Data Science Handbook*. Sebastopol, CA, USA: O'Reilly Media, 2016.
- [23] J. Brownlee, *XGBoost with Python: Gradient Boosted Trees with XGBoost and scikitlearn*. Machine Learning Mastery, 2016.
- [24] Kaggle, “Autism Screening Dataset,” Available: <https://www.kaggle.com>